INGÉNIERIE DES TESTS ET VALIDATIONS

Rapport de projet — Qualité logicielle

Rapport de Test

EasyVote

Plateforme Web de Vote Universitaire

Plan de test, gestion des tests avec Testiny, exécution du plan et organisation du travail d'équipe avec Jira

Réalisé par

Talel Mejri
Rawen Zgarni
Issra Elakrout

Module

Ingénierie des Tests et Validations

Enseignant

Dr. Ing. Faouzi Jaidi

Octobre 2026

Année universitaire 2026–2027

Résumé

EasyVote est une plateforme web de vote universitaire dont l'exigence centrale est l'**anonymat cryptographique** du scrutin. Ce rapport décrit la démarche de test de l'équipe : expression de 15 exigences traçables, plan de 68 cas de test sur quatre niveaux (unitaire, intégration, système, acceptation), puis exécution avec des techniques statiques et dynamiques, manuelles et automatisées. Le travail d'équipe est organisé avec **Jira** (méthode Scrum) et les tests sont gérés dans **Testiny**.

Table 1 – Chiffres clés de l'exécution

Indicateur	Valeur
Exigences identifiées	15
Cas de test du plan	68
Tests JUnit automatisés	38 : 36 réussis, 2 en échec (défauts détectés)
Scénario de démonstration de bout en bout	12 vérifications sur 12 réussies
Défauts détectés	9 : 1 critique, 5 majeurs, 3 mineurs
Couverture de code (JaCoCo)	100 % des branches de la génération des jetons ; 38,0 % des lignes du projet

Conclusion principale. Le scénario de démonstration fonctionne, mais l'**anonymat du vote n'est pas garanti** : une jointure SQL permet de relier un électeur à son vote (DEF-00, défaut critique). Sa correction est la priorité absolue, suivie des défauts de sécurité DEF-05 et DEF-06.

Table des matières

Résumé **i**

Abréviations **viii**

Introduction **1**

1 Choix et Présentation du Projet **2**

1.1 Identité du Projet . . . 2

1.2 Contexte et Objectifs . . . 2

1.3 Pourquoi EasyVote comme support de test . . . 2

1.4 Parties Prenantes . . . 3

1.5 Code Source du Projet . . . 3

2 Organisation du Travail d'Équipe avec Jira **4**

2.1 Pourquoi un outil de gestion de projet . . . 4

2.2 Mise en place du projet Jira . . . 4

2.3 Sprint 1 : backlog et tableau . . . 5

2.4 Rôle de chaque outil . . . 6

3 Besoins Fonctionnels et Non Fonctionnels **7**

3.1 Besoins Fonctionnels . . . 7

3.1.1 Étudiant . . . 7

3.1.2 Candidat . . . 8

3.1.3 Administrateur . . . 8

3.2 Besoins Non Fonctionnels . . . 8

3.3 Exigences identifiées et traçabilité . . . 9

4 Outil de Gestion des Tests : Testiny **10**

4.1 Choix de l'outil . . . 10

4.2 Justification du choix . . . 11

4.3 Configuration du projet EasyVote dans Testiny 11

5 Plan de Test **13**

5.1 Stratégie de test 13
5.2 Niveaux et types de tests 13
5.3 Outils de test retenus 14
5.4 Cas de test par niveau 14
5.4.1 Niveau Unitaire 14
5.4.2 Niveau Intégration 15
5.4.3 Niveau Système (E2E) 17
5.4.4 Niveau Acceptation (UAT) 18

6 Exécution du Plan de Test **20**

6.1 Organisation de l'exécution 20
6.2 Techniques statiques 21
6.2.1 Place des techniques statiques dans le projet 21
6.2.2 Revue technique 21
6.2.3 Analyse statique avec SonarQube (backend) 22
6.2.4 Analyse statique outillée du frontend 23
6.3 Techniques dynamiques boîte noire 23
6.3.1 Partition d'équivalence — code OTP 23
6.3.2 Analyse des valeurs limites — candidature 24
6.3.3 Table de décision — vote d'un étudiant 24
6.3.4 Test de transition d'état — cycle de vie d'une élection 24
6.3.5 Test des cas d'utilisation — UC-03 « Voter » 25
6.4 Techniques dynamiques boîte blanche 25
6.4.1 Couverture des instructions — `hashToken()` 25
6.4.2 Couverture des branches — `generateTokensForElection()` 25
6.4.3 Couverture des décisions — `validateDates()` 26
6.4.4 Complexité cyclomatique — `castVote()` 26
6.4.5 Couverture MC/DC — fin de la période de vote 27
6.4.6 Couverture mesurée par JaCoCo 27
6.5 Tests unitaires : chatbot et non-régression 28
6.6 Tests d'intégration et tests d'API 28
6.6.1 Tests d'intégration automatisés (JUnit + MockMvc + H2) 28
6.6.2 Tests d'API avec Postman 29

6.6.3 Vérification manuelle du JWT avec jwt.io (JWT-01) 32
6.7 Tests système de bout en bout 32
6.8 Tests de charge et de performance avec JMeter 33
6.9 Tests manuels 34
6.9.1 Vérification SQL de l'anonymat (SQL-01) 34
6.9.2 Tests exploratoires 35
6.9.3 Tests d'acceptation automatisés 35
6.9.4 Tests d'acceptation manuels (UAT) 35
6.9.5 Test de maintenance : sauvegarde et restauration 36
6.9.6 Tests liés aux changements : confirmation et régression 36
6.10 Tests manuels et automatisés : synthèse 36
6.11 Bilan de l'exécution 37
6.11.1 Défauts détectés 37

Conclusion **39**

Liste des tableaux

1 Chiffres clés de l'exécution i
2 Objectifs du travail de test et organisation du rapport 1

1.1 Parties prenantes du projet EasyVote 3

2.1 Répartition des rôles entre les outils de l'équipe 6

3.1 Matrice de traçabilité exigences → tests exécutés 9

4.1 Comparaison des outils de gestion des tests évalués (appréciation qualitative de l'équipe, sans mesure de performance) 10

5.1 Risques métier principaux et réponse de test 13
5.2 Répartition des 68 cas du plan par niveau et par type 13
5.3 Outils de test par niveau, type et technique 14
5.4 Plan de test — Niveau Unitaire 15
5.5 Plan de test — Niveau Intégration 15
5.6 Plan de test — Niveau Système 17
5.7 Plan de test — Niveau Acceptation 18

6.1 Outils utilisés pendant l'exécution 20
6.2 Techniques statiques du cours et application dans EasyVote 21
6.3 Résultats de la revue technique du code 21
6.4 Résultats de l'analyse SonarQube 22
6.5 Résultats de l'analyse statique du frontend 23
6.6 Partitions d'équivalence du code OTP 24
6.7 Valeurs limites des champs de candidature 24
6.8 Table de décision du vote 24
6.9 Transitions d'état de l'élection 24
6.10 Cas d'utilisation UC-03 : Voter 25
6.11 Couverture des instructions de `hashToken()` 25
6.12 Couverture des branches de la génération des jetons 26

6.13 Couverture des décisions de la validation des dates 26
6.14 Chemins indépendants de `castVote()` 27
6.15 Couverture MC/DC de la condition de fin de vote 27
6.16 Couverture de code mesurée (JaCoCo) 27
6.17 Tests unitaires du chatbot EasyBot et de non-régression 28
6.18 Tests d'intégration exécutés 29
6.19 Tests Postman 29
6.20 Résultats JMeter selon le nombre d'utilisateurs simultanés 33
6.21 Tests exploratoires 35
6.22 Tests d'acceptation automatisés 35
6.23 Exécution des tests d'acceptation 35
6.24 Répartition des tests exécutés par technique et par mode 36
6.25 Bilan chiffré de l'exécution 37
6.26 Défauts détectés pendant l'exécution 37

Table des figures

2.1 Jira — backlog du sprint 1 : tickets de l'epic « Tests boîte noire », avec statut, priorité et responsable 5

2.2 Jira — tableau Scrum du sprint 1 : colonnes To Do, In Progress, In Review et Done 5

4.1 Testiny — projet easyvote : 68 cas de test répartis en 4 dossiers (niveaux), dossier Système (E2E) ouvert 12

6.1 SonarQube — projet EasyVote : Quality Gate réussi, 0 vulnérabilité, 3 bugs, 45 code smells, couverture 35,3 %, duplication 2,7 % 23

6.2 Rapport JaCoCo — couverture par paquet : 39 % des instructions et 25 % des branches (1 636 lignes, 38 % couvertes) 28

6.3 Postman — PM-01 : `POST /api/auth/login` avec le compte administrateur, 200 OK ; le jeton d'accès, le jeton de rafraîchissement et `expiresIn` (86 400 000 ms, soit 24 h) sont renvoyés 30

6.4 Postman — PM-03 : `GET /api/auth/me` avec le jeton en *Bearer Token*, 200 OK en 96 ms ; le profil est celui du jeton et le mot de passe n'est pas renvoyé 31

6.5 Postman — `GET /api/admin/users` avec le jeton ADMIN, 200 OK en 41 ms : la contrepartie de PM-02, qui renvoie 401 sans jeton 31

6.6 jwt.io — access token décodé : algorithme HS512, rôle ADMIN, `iat` = 1792444682 et `exp` = 1792531082 (écart de 86 400 s, soit 24 h) 32

6.7 Rapport JMeter — scénario à 200 utilisateurs : 2 200 requêtes, 0 erreur, temps moyen 13,5 ms, 90e centile 19,9 ms 34

Abréviations

Sigle	Signification
CDC	Cahier des charges
EX-xx	Exigence identifiée
DEF-xx	Défaut détecté
UT, IT, ST, UAT	Tests unitaire, d'intégration, système, d'acceptation (*User Acceptance Test*)
REG, ...-Mxx	Test de régression (ou de maintenance)
MC/DC	*Modified Condition/Decision Coverage*
V(G)	Complexité cyclomatique
JWT	*JSON Web Token*
OTP	*One-Time Password*, code à usage unique
RBAC	*Role-Based Access Control*, contrôle d'accès par rôle
SaaS	*Software as a Service*

Introduction

Ce rapport présente le travail de test réalisé sur la plateforme **EasyVote**, application web de vote universitaire destinée aux étudiants, aux candidats et à l'administration. Il répond, dans l'ordre, aux cinq objectifs suivants :

Table 2 – Objectifs du travail de test et organisation du rapport

N°	Objectif	Où dans ce rapport
1	Choisir le projet	Chapitre 1
2	Présenter les besoins fonctionnels et non fonctionnels	Chapitre 3
3	Utiliser un outil de gestion des tests et des exigences	Chapitre 4 (Testiny)
4	Élaborer un plan de test (niveaux et types)	Chapitre 5
5	Exécuter le plan (techniques statiques et dynamiques, tests manuels et automatisés)	Chapitre 6

Une attention particulière est portée à l'**anonymat cryptographique du vote**, exigence centrale et différenciante du projet. L'exécution a d'ailleurs révélé que cette exigence n'est pas respectée par le code actuel (défaut critique DEF-00, section 6.9.1).

Le travail a été réparti entre les trois membres de l'équipe avec **Jira** (chapitre 2), tandis que **Testiny** (chapitre 4) gère les cas de test, les exécutions et leurs résultats.

Chapitre 1

Choix et Présentation du Projet

1.1 Identité du Projet

EasyVote est une plateforme web de vote universitaire qui digitalise l'intégralité du processus électoral : phase de campagne (présentation des candidats et engagement avec les électeurs), phase de vote (scrutin sécurisé et anonyme) et phase post-électorale (publication des résultats). La plateforme s'appuie sur **Spring Boot 3.3.5 (Java 21)**, **React + TypeScript** et **MySQL/MariaDB**, et intègre un assistant virtuel hybride (règles + IA optionnelle) ainsi qu'un centre de notifications.

1.2 Contexte et Objectifs

Les élections étudiantes reposent encore largement sur des processus traditionnels (bulletins papier, urnes physiques, dépouillement manuel), ce qui entraîne une faible participation (35–40 %), une lourdeur organisationnelle (10 à 15 jours de préparation) et un manque de transparence. Le projet poursuit cinq objectifs principaux :

1. Digitaliser le processus électoral de bout en bout.
2. Garantir l'anonymat et l'intégrité du vote grâce à une architecture cryptographique séparant l'identité de l'électeur de son bulletin.
3. Augmenter la participation électorale par une expérience moderne, accessible depuis tout appareil.
4. Favoriser l'engagement civique (likes, commentaires, questions/réponses, annonces).
5. Démontrer la maîtrise des principes du génie logiciel (architecture en couches, sécurité, tests, documentation).

1.3 Pourquoi EasyVote comme support de test

EasyVote est un bon support de test car il réunit des exigences variées et vérifiables :

— des **règles métier précises** (cycle de vie d'une élection, vote unique, limites de saisie), idéales pour les techniques boîte noire ;

— un **cœur algorithmique** (jetons SHA-256, méthode de vote) adapté aux techniques boîte blanche ;

— des **exigences non fonctionnelles** mesurables (temps de réponse, sécurité JWT, contrôle d'accès) ;

— une exigence **critique** (anonymat) dont l'échec aurait un impact majeur, ce qui justifie une approche de test basée sur les risques.

1.4 Parties Prenantes

Table 1.1 – Parties prenantes du projet EasyVote

Acteur	Rôle	Responsabilités
Étudiants (`STUDENT`)	Public principal	Consulter les élections, voter, liker, commenter, poser des questions, postuler comme candidat
Candidats (`CANDIDATE`)	Acteurs de campagne	Gérer leur programme, publier des annonces, répondre aux questions, consulter les statistiques d'engagement
Administrateur (`ADMIN`)	Superviseur	Créer/gérer les élections, valider les candidatures, générer les jetons, suivre les statistiques temps réel (ne vote pas)

1.5 Code Source du Projet

Le code source (backend Spring Boot, frontend React, scripts de configuration et de seed) ainsi que tous les tests réalisés (dossiers `easyvote-backend/src/test` et `tests/`) sont versionnés sur le dépôt : `https://github.com/LtayefAhmed/EasyVote`

Chapitre 2

Organisation du Travail d'Équipe avec Jira

2.1 Pourquoi un outil de gestion de projet

Le projet est réalisé par trois personnes qui travaillent en parallèle sur le code, les tests et le rapport. Sans outil commun, le risque est de dupliquer un travail, d'oublier un test ou de ne plus savoir qui fait quoi. L'équipe a donc adopté **Jira**, outil de gestion de projet agile, avec la méthode **Scrum** : un backlog de tâches priorisées, un sprint de durée fixe et un tableau d'avancement partagé.

2.2 Mise en place du projet Jira

— Un espace **Easy Vote** de type Scrum, partagé par les trois membres : Talel Mejri (TM), Rawen Zgarni (RZ) et Issra Elakrout (IE). Chaque ticket est assigné à un membre.

— **Cinq epics**, un par famille de tests : boîte noire, boîte blanche, unitaires, intégration et acceptation.

— Des **tâches**, une par test automatisé (38 au total), et des **bugs** pour les défauts DEF-xx trouvés pendant l'exécution.

— Un backlog **importé en une seule fois** depuis le fichier `tests/backlog-jira.csv`, généré à partir de la liste des tests du script `lancer-tests.ps1`. Chaque ticket reprend l'identifiant du plan (PE-01, R1, IT-04…), la classe et la méthode JUnit, la commande de lancement et le résultat attendu : un ticket Jira correspond directement à un test exécutable.

— Un **workflow** à quatre états : *To Do*, *In Progress*, *In Review*, *Done*. L'état *In Review* prévoit la relecture du travail par un autre membre avant la clôture, ce qui correspond à la revue entre pairs du cours (section 6.2.1).

— Une **priorité** (de *Highest* à *Low*) fixée selon le risque métier, comme dans la stratégie du chapitre 5, et des **étiquettes** indiquant le type et le niveau du test.

Définition du « terminé ». Un ticket passe en *Done* lorsque le test est écrit, exécuté (`lancer-tests.ps1 <identifiant>`), son résultat consigné et le travail relu par un autre

membre.

2.3 Sprint 1 : backlog et tableau

Le **sprint 1** (du 1er au 8 octobre) regroupe 11 tickets de l'epic « Tests boîte noire ». La figure 2.1 montre le backlog du sprint (référence, résumé, epic, statut, priorité, responsable) et la figure 2.2 le tableau Scrum. Au moment de la capture, 5 tickets sont terminés (*Done*), 4 sont en relecture (*In Review*), 1 est en cours et 1 reste à faire.

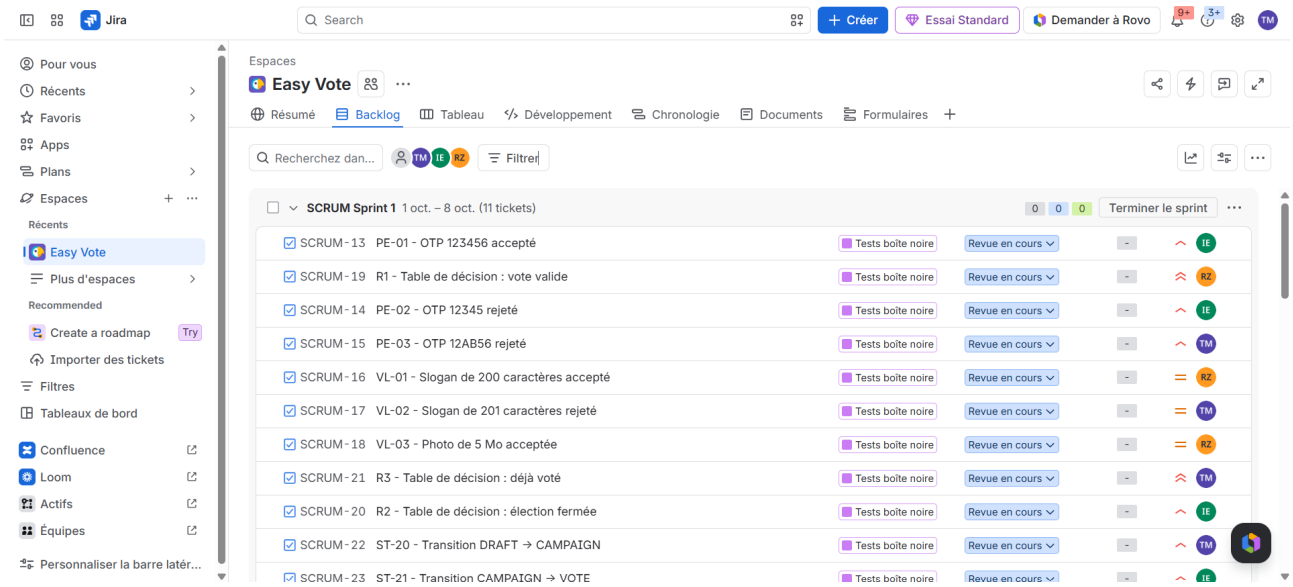


FIGURE 2.1 – Jira — backlog du sprint 1 : tickets de l'epic « Tests boîte noire », avec statut, priorité et responsable

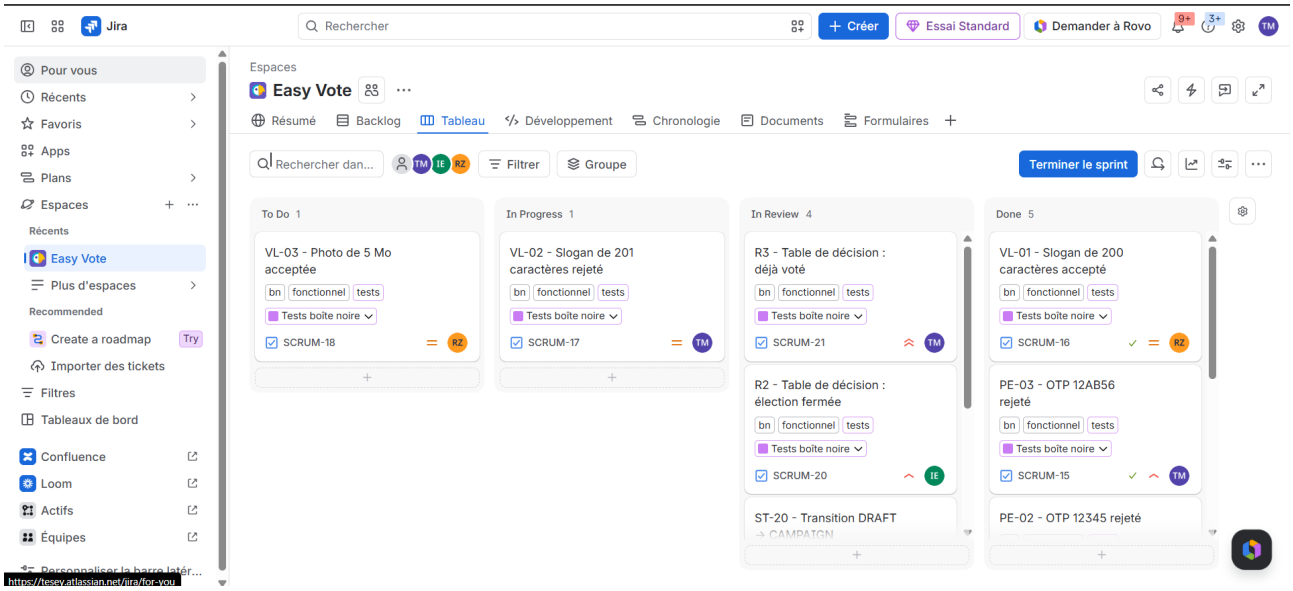


FIGURE 2.2 – Jira — tableau Scrum du sprint 1 : colonnes To Do, In Progress, In Review et Done

2.4 Rôle de chaque outil

Les outils se complètent : Jira répond à la question « qui fait quoi, et où en est-on ? », Testiny à « quels cas de test, avec quel résultat ? ».

TABLE 2.1 – Répartition des rôles entre les outils de l'équipe

Outil	Rôle	Contenu
Jira	Organisation du travail	Backlog, sprint, tableau d'avancement, bugs, responsables
Testiny	Gestion des tests	Cas de test, plans, exécutions, résultats, exigences (chapitre 4)
GitHub	Versionnement	Code source et tests automatisés
JUnit, scripts	Exécution des tests	Suite automatisée relançable en une commande

Chapitre 3

Besoins Fonctionnels et Non Fonctionnels

3.1 Besoins Fonctionnels

3.1.1 Étudiant

Compte et accès

— S'inscrire (nom, email, mot de passe) et activer son compte via un code OTP à 6 chiffres

— S'authentifier (JWT : access token 1 h, refresh token 7 jours)

Élections et candidatures

— Consulter les élections et la liste des candidats validés (slogan, programme)

— Postuler comme candidat lorsque l'élection est en statut `CAMPAIGN_ACTIVE` (slogan ≤ 200 caractères, programme ≤ 5000 caractères, photo JPG/PNG ≤ 5 Mo)

Vote

— Voter une seule fois lorsque l'élection est en statut `VOTE_ACTIVE`, avec étape de confirmation explicite

— Recevoir un code de confirmation à 10 caractères sans révéler son choix

— Consulter uniquement les statistiques de participation pendant le vote, puis les résultats détaillés après clôture

Engagement et accompagnement

— Liker un candidat (toggle), commenter, poser des questions

— Dialoguer avec le chatbot EasyBot

— Consulter et marquer comme lues ses notifications

3.1.2 Candidat

— Tous les droits de l'étudiant

— Modifier son programme et publier des annonces de campagne

— Répondre aux questions des étudiants

— Consulter ses statistiques d'engagement (tableau de bord « Ma campagne »)

3.1.3 Administrateur

— Créer, modifier, supprimer des élections et gérer leur cycle de vie (`DRAFT` → `CAMPAIGN_ACTIVE` → `VOTE_ACTIVE` → `CLOSED`)

— Valider ou rejeter les candidatures (avec note de rejet) ; le rôle passe alors de `STUDENT` à `CANDIDATE`

— Déclencher la génération (idempotente) des jetons de vote

— Consulter les statistiques détaillées temps réel (votes par heure, classement live)

— Générer et télécharger le rapport PDF officiel après clôture

— Gérer les utilisateurs

3.2 Besoins Non Fonctionnels

Performance. Chargement initial $< 2\,\text{s}$; latence API en lecture $< 200\,\text{ms}$; chatbot (règles) $< 50\,\text{ms}$ et réponse IA $< 3\,\text{s}$; rafraîchissement des notifications toutes les 30 s (paramétrable).

Sécurité. Authentification JWT stateless ; mots de passe hachés (BCrypt, cost factor 10) ; endpoints protégés par `@PreAuthorize` (hiérarchie ADMIN > CANDIDATE > STUDENT) ; validation systématique des entrées (Bean Validation, react-hook-form) ; CORS strict ; mots de passe jamais retournés ni journalisés.

Anonymat et intégrité du vote. Aucune référence à `user_id` dans la table `votes` ; jeton haché en SHA-256, seul le hash étant stocké ; aucune requête SQL ne doit pouvoir relier un électeur à son vote ; un seul vote par électeur ; génération des jetons idempotente.

Maintenabilité. Architecture en couches (Controller / Service / Repository) ; gestion centralisée des erreurs (`GlobalExceptionHandler`) ; conventions de nommage cohérentes.

Accessibilité et ergonomie. Interface responsive mobile-first ; mode sombre/clair persistant ; composants shadcn/ui (Radix UI, ARIA) ; animations respectant `prefers-reduced-motion` ; français en V1.

Évolutivité. Backend stateless scalable horizontalement ; ajout d'un fournisseur IA pour le chatbot par simple configuration.

3.3 Exigences identifiées et traçabilité

Pour permettre la gestion des exigences dans l'outil de test, les besoins ont été regroupés en 15 exigences identifiées (`EX-xx`). Chaque cas de test exécuté est rattaché à au moins une exigence.

Table 3.1: Matrice de traçabilité exigences → tests exécutés

ID	Exigence	Source CDC	Tests exécutés	Verdict
EX-01	Inscription et activation par code OTP à 6 chiffres	§4.4.1, UC-01	PE-01..03, IT-01	**Réussi**
EX-02	Authentification JWT (access 1 h, refresh 7 j)	§4.4.1	PM-01, PM-03, JWT-01	**Échec**
EX-03	Contrôle d'accès par rôle ; l'admin ne vote pas	§4.4.1, §6.1	IT-09, PM-02, EX-3	**Échec**
EX-04	Cycle de vie d'une élection	§3.1.1	ST-20, ST-21, ST-23, DC-01..03	**Réussi**
EX-05	Candidature : statut, slogan ≤ 200, photo ≤ 5 Mo	§3.1.2	VL-01..03, EX-1	**Échec**
EX-06	Validation de candidature et passage au rôle CANDIDATE	§3.1.2	ST-01, EX-2	**Échec**
EX-07	Génération idempotente des jetons (SHA-256)	§3.2.1	UT-01a..c, B1..B3	**Réussi**
EX-08	Un seul vote par électeur	§3.2.2	R1..R3, ST-02	**Réussi**
EX-09	Anonymat : aucun lien SQL électeur $\leftrightarrow$ vote	§3.2.2, §4.4.2	IT-04, SQL-01, CC-3	**Échec**
EX-10	Code de confirmation à 10 caractères	§3.2.2	R1, ST-01	**Réussi**
EX-11	Transparence des résultats selon le statut	§3.2.3	ST-03	**Réussi**
EX-12	Export du rapport PDF officiel	§3.2.4	ST-05	**Réussi**
EX-13	Chatbot : réponses par règles en < 50 ms	§3.3.1, §5.1	UT-01 (chatbot), UT-06, ex-UT-CB3	**Échec**
EX-14	Latence API en lecture < 200 ms	§5.1	ST-NF, PM-02, ST-08 (JMeter)	**Échec**
EX-15	Sauvegarde et restauration des données	§5.5	MT-01, MT-02	**Réussi**

Chapitre 4

Outil de Gestion des Tests : Testiny

4.1 Choix de l'outil

Avant de retenir Testiny, l'équipe a évalué deux outils open source de gestion des tests, **TestLink** puis **Kiwi TCMS**, tous deux à installer sur un serveur local. Elle a finalement retenu **Testiny**, outil en ligne (SaaS), pour gérer à la fois les cas de test, les plans, les exécutions et leur rattachement aux exigences. Le tableau 4.1 résume la comparaison.

TABLE 4.1 – Comparaison des outils de gestion des tests évalués (appréciation qualitative de l'équipe, sans mesure de performance)

Critère	**TestLink**	**Kiwi TCMS**	**Testiny**
Licence / offre	Open source (GPL)	Open source (GPL)	SaaS ; offre gratuite pour une petite équipe
Déploiement	Serveur local : Apache, PHP, MySQL	Serveur local : Docker, MariaDB, HTTPS	Aucun : navigateur web
Mise en route	Longue (installation, configuration)	Moyenne (conteneurs, certificat)	Immédiate (compte en ligne)
Interface	Datée	Moderne mais dense	Moderne et épurée
Travail à trois	Comptes à créer sur notre serveur	Comptes à créer sur notre serveur	Invitation par email, accès simultané
Exigences	Module d'exigences intégré	Liens vers les anomalies, pas de module d'exigences dédié	Exigence notée dans chaque cas (EX-xx)
Intégration Jira	Oui (suivi d'anomalies)	Oui (suivi d'anomalies)	Oui
Résultats automatisés	API XML-RPC	API JSON-RPC et XML-RPC	Module *Automation* (API)
Entretien de l'outil	À notre charge (mises à jour, sauvegardes)	À notre charge	Aucun
Coût pour le projet	Gratuit	Gratuit	Gratuit
Décision	**Écarté**	**Écarté**	**Retenu**

Les deux outils open source sont complets mais demandent d'installer et d'entretenir un serveur, ce qui n'apporte rien à un projet de trois personnes. Testiny offre le même cycle (cas, plans, exécutions, résultats) sans aucune installation.

4.2 Justification du choix

— **Aucune installation** : application web accessible depuis un navigateur, contrairement à Kiwi TCMS qui nécessite Docker, une base MariaDB et un certificat local.

— **Travail collaboratif** : les trois membres travaillent en même temps sur le même projet, chacun avec son compte.

— **Organisation claire** : cas de test rangés en dossiers (un par niveau de test), avec identifiants automatiques (`TC-xx`, `TCF-x`).

— **Cycle complet** : cas de test → plans de test → exécutions (*test runs*) → résultats (Passed, Failed, Blocked, Skipped, Not Run).

— **Suivi** : tableau de bord, jalons (*milestones*) et module *Automation* pour rattacher des résultats de tests automatisés.

— **Traçabilité** : chaque cas de test contient l'identifiant de l'exigence couverte (`EX-xx`) et les défauts sont liés aux exécutions en échec.

4.3 Configuration du projet EasyVote dans Testiny

1. Création de l'organisation et du projet **easyvote**, et invitation des trois membres de l'équipe.
2. Création d'un dossier par niveau de test, reprenant le plan du chapitre 5 : **Unitaire** (12 cas), **Intégration** (24 cas), **Système (E2E)** (22 cas), **Acceptation (UAT)** (10 cas), soit **68 cas de test**.
3. Saisie de chaque cas : titre préfixé par sa référence (UT-01, IT-04, ST-01…), préconditions, étapes, résultat attendu, priorité et exigence couverte.
4. Création des **test runs** (un par niveau) pour l'exécution, puis saisie du résultat de chaque cas et du défaut associé en cas d'échec.

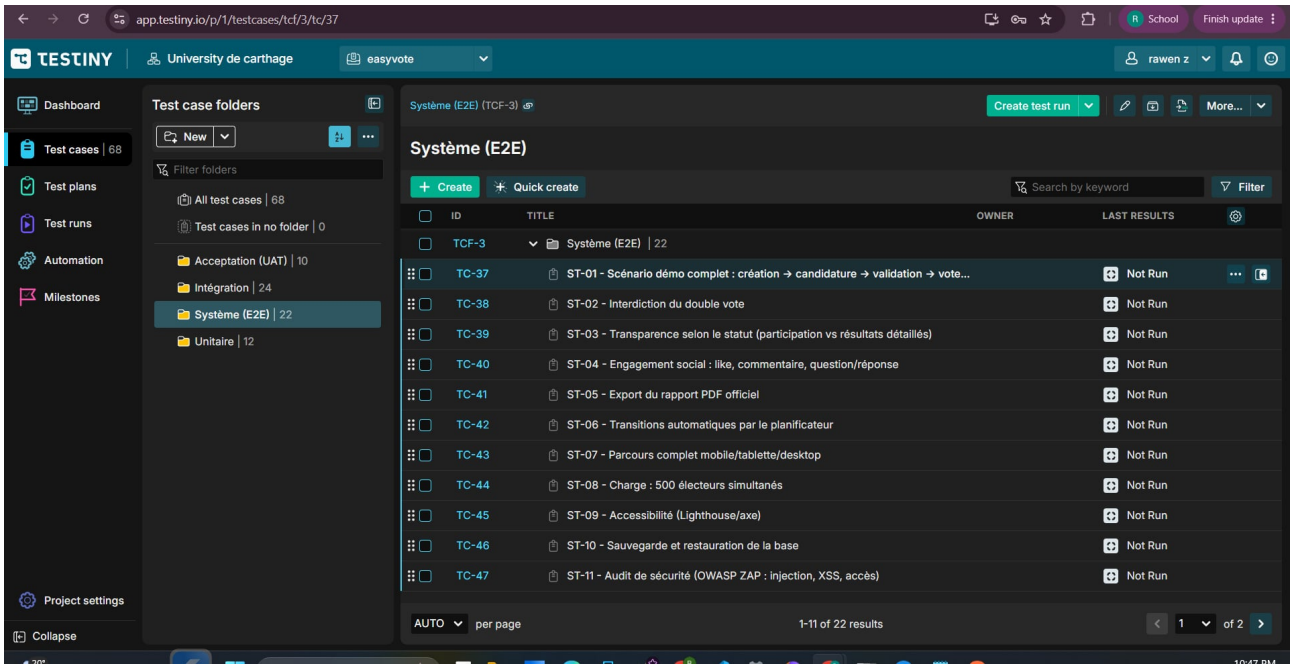


FIGURE 4.1 – Testiny — projet easyvote : 68 cas de test répartis en 4 dossiers (niveaux), dossier Système (E2E) ouvert

Chapitre 5

Plan de Test

5.1 Stratégie de test

La stratégie est **basée sur les risques** : les fonctions dont l'échec aurait l'impact le plus fort (anonymat, vote unique, contrôle d'accès) reçoivent la priorité *Critique* et sont testées en premier, à plusieurs niveaux et avec plusieurs techniques.

Table 5.1 – Risques métier principaux et réponse de test

Risque	Niveau	Réponse de test
Un électeur peut être relié à son vote	Critique	IT-04, SQL-01, revue du modèle de données
Un étudiant vote deux fois	Critique	Table de décision R1..R3, ST-02
Un utilisateur accède à une fonction d'un autre rôle	Critique	IT-09, PM-02, test exploratoire EX-3
Résultats divulgués pendant le vote	Haute	ST-03
Saisies invalides acceptées (OTP, slogan, photo)	Haute	Partitions d'équivalence, valeurs limites
Lenteur de l'API ou du chatbot	Moyenne	UT-06, ST-NF, PM-02

5.2 Niveaux et types de tests

Le plan couvre les **quatre niveaux** (unitaire, intégration, système, acceptation) et les **types** suivants : **fonctionnels** (le système fait-il ce qui est attendu ?), **non fonctionnels** (performance, sécurité, utilisabilité), **structurels** (couverture du code), **liés aux changements** (tests de confirmation et de régression) et de **maintenance** (sauvegarde, migration, mise à jour).

Table 5.2 – Répartition des 68 cas du plan par niveau et par type

Niveau	Fonct.	Non-fonct.	Struct.	Régr. / Maint.	Total
Unitaire	5	2	2	3	12
Intégration	11	3	4	6	24
Système (E2E)	7	4	1	10	22
Acceptation (UAT)	4	2	1	3	10
Total	27	11	8	22	**68**

5.3 Outils de test retenus

Quatre outils principaux couvrent les niveaux et les types du plan. Ils sont complétés par des outils légers pour les vérifications ponctuelles.

Table 5.3 – Outils de test par niveau, type et technique

Outil	Niveaux	Types	Technique et utilisation
JUnit 5 (+ Mockito, JaCoCo)	Unitaire, intégration	Fonctionnel, structurel, régression	Dynamique, boîte noire et boîte blanche ; couverture mesurée par JaCoCo ; automatisé
Postman (+ Newman)	Intégration, système	Fonctionnel, non fonctionnel (sécurité)	Dynamique, boîte noire sur l'API REST ; collection rejouable ; automatisé
SonarQube	Code (tous niveaux)	Structurel, maintenabilité, sécurité	Statique : bugs, vulnérabilités, *code smells*, complexité, duplication ; automatisé
JMeter	Système	Non fonctionnel (performance, charge)	Dynamique : utilisateurs simultanés, assertions de temps de réponse ; automatisé
jwt.io, SQL, navigateur	Intégration, système, acceptation	Fonctionnel, sécurité, utilisabilité	Vérifications manuelles (jeton, anonymat, exploratoire, UAT)
ESLint, tsc, npm audit	Code frontend	Structurel, sécurité	Statique outillée
Testiny	Tous	Tous	Gestion des cas, exécutions, résultats et défauts
Jira	Tous	—	Organisation du travail : backlog, sprint, tableau, bugs (chapitre 2)

Critères d'entrée de l'exécution. Application compilée et démarrée (backend, base MariaDB, frontend), jeu de données de test chargé (`tests/seed-demo.sql`), cas de test saisis dans Testiny.

Critères de sortie. Tous les cas *Critique* exécutés ; chaque échec enregistré comme défaut et rattaché à son cas de test ; aucune régression sur les tests déjà réussis.

5.4 Cas de test par niveau

Chaque niveau est détaillé selon les colonnes : Référence, Type (Fonct. = Fonctionnel, Non-fonct. = Non-fonctionnel, Struct. = Structurel, Régr. = Régression / maintenance), Scénario, Priorité, Données de test, Résultat attendu et **Résultat** de l'exécution (chapitre 6) : **Réussi**, **Échec**, **Partiel** (exécuté en partie) ou Non exécuté.

5.4.1 Niveau Unitaire

Table 5.4: Plan de test — Niveau Unitaire

Réf.	Type	Scénario	Prio.	Données	Résultat attendu	Résultat
UT-01	Fonct.	`hashToken()` : SHA-256 d'un jeton	Critique	jeton fixe, vecteur NIST	Hash de 64 caractères hexadécimaux, déterministe, différent du jeton	**Réussi**
UT-02	Fonct.	Génération du code de confirmation	Haute	vote valide	Code de 10 caractères, sans doublon	**Partiel**
UT-03	Fonct.	Hachage du mot de passe (BCrypt)	Moyenne	mdp = `Test@1234`	Hash ≠ clair, préfixe `$2a$`	**Réussi**
UT-04	Fonct.	Génération et expiration du code OTP	Haute	code généré à t, test à t+9 et t+11 min	6 chiffres ; valide à 9 min, expiré à 11 min	Non exécuté
UT-05	Fonct.	Décodage d'un token JWT valide	Critique	token réel décodé (jwt.io)	Rôle et expiration conformes (access 1 h / refresh 7 j)	**Échec**
UT-06	Non-fonct.	Temps de réponse du chatbot (couche règles)	Moyenne	question « Comment voter ? »	< 50 ms	**Réussi**
UT-07	Non-fonct.	Complexité cyclomatique de `castVote()`	Basse	SonarQube + calcul de V(G)	≤ seuil défini (10)	**Échec**
UT-08	Struct.	Couverture des branches des transitions d'état	Haute	2 transitions valides + 1 illégale	100 % des branches couvertes	**Partiel**
UT-09	Struct.	Exceptions métier du vote	Haute	déjà voté, élection non active	Exceptions métier levées, aucun vote enregistré	**Réussi**
UT-M01	Régr.	Non-régression du calcul des résultats après refactoring	Haute	jeu de votes de référence	Résultat identique avant/après	**Réussi**
UT-M02	Régr.	Compatibilité ascendante des DTO	Moyenne	ancien payload JSON	Désérialisation OK, valeur par défaut	Non exécuté
UT-M03	Régr.	Couverture de tests après modification d'un service	Haute	rapport JaCoCo avant/après	Couverture maintenue ≥ 70 %	Non exécuté

Remarques. UT-02 : le format (10 caractères `[A-Z0-9]`) est vérifié par R1, pas l'absence de doublon. UT-05 : le jeton dure 24 h (DEF-06). UT-07 : V(G) = 12 > 10. UT-08 : la transition VOTE_ACTIVE → CLOSED n'a pas été testée. UT-M01 : exécuté en JUnit (`RegressionResultatsTest`) sur un jeu de 10 votes de référence ; il sert de filet de sécurité lors d'un refactoring.

5.4.2 Niveau Intégration

Table 5.5: Plan de test — Niveau Intégration

Réf.	Type	Scénario	Prio.	Données	Résultat attendu	Résultat
IT-01	Fonct.	`POST /auth/register` ↔ email OTP ↔ BDD	Haute	nom, email, mdp valides	Compte `STUDENT` non vérifié créé, email OTP envoyé	**Réussi**
IT-02	Fonct.	`POST /auth/login` ↔ MariaDB	Haute	utilisateur vérifié + bon mdp	HTTP 200 + JWT (access + refresh)	**Réussi**

Réf.	Type	Scénario	Prio.	Données	Résultat attendu	Résultat
IT-03	Fonct.	Génération des jetons ↔ `vote_tokens` (idempotence)	Critique	élection + étudiants, 2 lancements	1 hash par électeur, 2e lancement sans effet	**Réussi**
IT-04	Fonct.	POST `/elections/{id}/vote` ↔ table `votes` (anonymat)	Critique	jeton valide, candidat validé	Pas de `user_id`, aucune jointure possible ; code retourné	**Échec**
IT-05	Fonct.	Chatbot ↔ API IA externe (fallback)	Moyenne	question hors FAQ	Réponse IA reçue et affichée	Non exécuté
IT-06	Fonct.	Validation d'une candidature ↔ rôle ↔ notification	Haute	candidature `PENDING`	Statut `VALIDATED`, rôle `CANDIDATE`, notification	**Réussi**
IT-07	Non-fonct.	Temps de réponse `GET /elections`	Moyenne	20 appels successifs	≤ 200 ms	**Réussi**
IT-08	Non-fonct.	Résilience si l'API IA est indisponible	Critique	timeout simulé	Repli sur les règles, aucune erreur 500	Non exécuté
IT-09	Struct.	Filtre JWT et `@PreAuthorize`	Critique	sans token ; `STUDENT` sur endpoint `ADMIN`	401 puis 403	**Réussi**
IT-10	Struct.	Contraintes d'unicité en base	Haute	2e like ; `token_hash` dupliqué	Insertions rejetées, erreur gérée	Non exécuté
IT-11	Fonct.	`POST /auth/refresh`	Haute	refresh token valide	Nouvel access token émis	Non exécuté
IT-12	Fonct.	`GET /elections` ↔ JPA ↔ MariaDB	Haute	élections en base	Liste paginée, tri par date	**Partiel**
IT-13	Fonct.	Upload photo de candidature ↔ stockage	Haute	photo JPG ≤ 5 Mo	Fichier stocké, URL retournée	Non exécuté
IT-14	Fonct.	`GET /results` ↔ agrégation SQL	Critique	votes en base	Totaux et pourcentages exacts	**Réussi**
IT-15	Fonct.	Marquer une notification comme lue	Moyenne	notification existante	Statut `read`, badge décrémenté	Non exécuté
IT-16	Non-fonct.	Rollback sur vote échoué	Critique	erreur simulée après insert	Aucune ligne partielle, jetons intacts	Non exécuté
IT-17	Struct.	Contrainte FK candidat → élection	Moyenne	suppression élection avec candidats	Cascade ou rejet selon règle	Non exécuté
IT-18	Struct.	Index sur `votes`	Moyenne	EXPLAIN sur requête	Index utilisés	Non exécuté
IT-M01	Régr.	Migration de schéma (ajout de colonne)	Haute	script + base peuplée	Pas de perte de données	Non exécuté
IT-M02	Régr.	Rollback de migration	Critique	échec simulé	Retour à l'état antérieur	Non exécuté
IT-M03	Régr.	Nouveau format de réponse de l'API IA	Haute	mock nouvelle réponse	Parsing adapté	Non exécuté
IT-M04	Régr.	Montée de version Spring Boot	Moyenne	benchmark avant/après	Aucune dégradation	Non exécuté
IT-M05	Régr.	Compatibilité des endpoints après ajout	Haute	collection Postman rejouée	Aucun breaking change	Non exécuté

IT-M06	Régr.	Non-régression de l'anonymat	Critique	schéma + données après vote	Aucune colonne ni jointure vers `user_id`	**Échec**

Remarques. IT-02 et IT-07 ont été exécutés via Postman et le script système. IT-06 : l'état final est correct mais le rôle change trop tôt (DEF-02, test exploratoire EX-2). IT-12 : la liste est triée mais non paginée. IT-05 et IT-08 : fournisseur IA désactivé dans la configuration.

5.4.3 Niveau Système (E2E)

TABLE 5.6: Plan de test — Niveau Système

Réf.	Type	Scénario	Prio.	Données	Résultat attendu	Résultat
ST-01	Fonct.	Scénario démo : création → candidature → validation → vote → résultats	Critique	admin, 2 étudiants, 1 élection	Parcours sans erreur, résultats corrects	**Réussi**
ST-02	Fonct.	Interdiction du double vote	Critique	étudiant ayant déjà voté	2e vote refusé, message clair	**Réussi**
ST-03	Fonct.	Transparence selon le statut	Critique	élection `VOTE_ACTIVE` puis `CLOSED`	Participation seule pendant le vote ; détails après	**Réussi**
ST-04	Fonct.	Engagement social : like, commentaire, Q/R	Moyenne	étudiant + candidat	Actions visibles, notifications	Non exécuté
ST-05	Fonct.	Export du rapport PDF officiel	Haute	élection `CLOSED`	PDF valide et complet	**Réussi**
ST-06	Fonct.	Transitions automatiques (planificateur)	Haute	dates proches	Statuts mis à jour	Non exécuté
ST-07	Fonct.	Parcours mobile / tablette / desktop	Moyenne	3 résolutions	Affichage cohérent	Non exécuté
ST-08	Non-fonct.	Charge : 500 électeurs simultanés	Haute	scénario JMeter	$\leq$ 200 ms, aucune erreur	**Échec**
ST-09	Non-fonct.	Accessibilité (Lighthouse/axe)	Moyenne	pages clés	Aucune erreur bloquante	Non exécuté
ST-10	Non-fonct.	Sauvegarde et restauration de la base	Haute	sauvegarde + restauration	Données restaurées à l'identique	**Réussi**
ST-11	Non-fonct.	Audit de sécurité	Haute	SonarQube (code) ; OWASP ZAP (API)	Aucune vulnérabilité critique	**Partiel**
ST-12	Struct.	Cohérence rôles/menus par profil	Haute	3 connexions	Interface conforme au rôle	Non exécuté
REG-00-1	Régr.	Connexion des 3 rôles (socle)	Critique	3 comptes de test	JWT et rôle corrects	**Réussi**
REG-00-2	Régr.	Workflow de vote (socle)	Critique	élection `VOTE_ACTIVE`	Vote enregistré, code	**Réussi**
REG-00-3	Régr.	Accès RBAC croisé (socle)	Critique	3 rôles × endpoints	403 si non autorisé	**Partiel**
REG-00-4	Régr.	Anonymat de la table `votes` (socle)	Critique	schéma + données	Aucune colonne ni jointure vers `user_id`	**Échec**
REG-00-5	Régr.	Temps de réponse des endpoints critiques	Haute	20 appels	$\leq$ 200 ms	**Réussi**

Réf.	Type	Scénario	Prio.	Données	Résultat attendu	Résultat
ST-M01	Régr.	Non-régression du module Vote après correctif	Critique	suite ST-01 à ST-03	Tous les scénarios passent	Non exécuté
ST-M02	Régr.	Déploiement sans interruption	Haute	rolling/blue-green	Disponibilité maintenue	Non exécuté
ST-M03	Régr.	Restauration après incident	Critique	sauvegarde + incident	Plateforme cohérente	Non exécuté
ST-M04	Régr.	Ajout d'un fournisseur IA	Moyenne	nouveau provider	Chatbot intact	Non exécuté
ST-M05	Régr.	Compatibilité multi-navigateurs	Moyenne	Chrome, Firefox, Safari	Rendu identique	Non exécuté

Remarques. ST-08 : à 500 utilisateurs, toutes les réponses sont correctes (HTTP 200) mais plus de la moitié dépassent 200 ms (DEF-08) ; à 200 utilisateurs l'exigence est respectée. ST-11 : le code a été audité par SonarQube (0 vulnérabilité) ; le scan dynamique OWASP ZAP n'a pas été réalisé. REG-00-3 : seul l'endpoint `/api/admin/users` a été testé, et l'admin peut voter (DEF-05). ST-M01 sera exécuté après correction des défauts.

5.4.4 Niveau Acceptation (UAT)

Table 5.7: Plan de test — Niveau Acceptation

Réf.	Type	Scénario	Prio.	Données	Résultat attendu	Résultat
UAT-01	Fonct.	Étudiant : connexion → consultation → vote	Critique	compte `talel@test.com`	Parcours compris, code reçu	**Réussi**
UAT-02	Fonct.	Candidat gère sa campagne (programme, annonces, réponses)	Haute	compte `achref@test.com`	Ergonomie validée	À exécuter
UAT-03	Fonct.	Administrateur crée une élection et traite les candidatures	Haute	compte `ahmed@test.com`	Processus clair, PDF obtenu	À exécuter
UAT-04	Fonct.	Pertinence des réponses du chatbot	Moyenne	10 questions d'étudiants	Réponses utiles et correctes	À exécuter
UAT-05	Non-fonct.	Perception de la rapidité	Moyenne	questionnaire post-usage	Note moyenne $\geq 4/5$	À exécuter
UAT-06	Non-fonct.	Clarté du PDF officiel	Moyenne	PDF réel	Validé comme exploitable	**Réussi**
UAT-07	Struct.	Rôles et droits conformes au besoin métier	Haute	test croisé des 3 rôles	Aucun accès incohérent	**Réussi**
UAT-M01	Régr.	Évolution demandée conforme au besoin	Haute	environnement de recette	Fonctionnalité conforme	À exécuter
UAT-M02	Régr.	Procédure de maintenance planifiée	Moyenne	fenêtre simulée	Utilisateurs informés	À exécuter
UAT-M03	Régr.	Rôles/droits inchangés après mise à jour	Haute	test croisé	Aucun changement non souhaité	**Réussi**

Remarques. UAT-01, UAT-06, UAT-07 et UAT-M03 ont été exécutés dans leur **version automatisée** (section 6.9.3). Les critères qui demandent un jugement humain (questionnaire, ergonomie, clarté du PDF) restent à valider manuellement, comme les autres cas d'acceptation.

Chapitre 6

Exécution du Plan de Test

6.1 Organisation de l'exécution

L'exécution combine des **techniques statiques** (revue, analyse outillée) et **dynamiques** (boîte noire, boîte blanche, exploratoire), des tests **automatisés** (rejouables en une commande) et **manuels**. Les résultats sont reportés dans les *test runs* de Testiny.

Table 6.1 – Outils utilisés pendant l'exécution

Outil	Usage
JUnit 5, AssertJ, Mockito	Tests unitaires boîte noire et boîte blanche (services isolés de la base)
Spring Boot Test, MockMvc, H2	Tests d'intégration : application démarrée, sécurité JWT, base en mémoire
JaCoCo	Mesure de la couverture du code (preuve des techniques boîte blanche)
Postman / Newman	Tests d'API (connexion, profil `/auth/me`, sécurité)
jwt.io	Décodage manuel du contenu des jetons JWT
Script Node.js	Tests système de bout en bout sur l'API réelle
Apache JMeter 5.6.3	Tests de charge et de performance (jusqu'à 500 utilisateurs simultanés)
SonarQube 10.6 (Docker)	Analyse statique de la qualité du code backend : bugs, vulnérabilités, *code smells*, complexité, duplication, couverture
ESLint, tsc, npm audit	Analyse statique du frontend
MariaDB (XAMPP)	Base réelle pour les tests système, SQL et maintenance
Testiny	Gestion des cas, des exécutions et des résultats
Jira	Organisation du travail de l'équipe : backlog, sprint, tableau, bugs

Tous les tests automatisés se relancent depuis la racine du projet :

```
.\tests\lancer-tests.ps1              # 38 tests JUnit (ou : boitenoire, boiteblanche,
    unitaire, integration, acceptation)
.\tests\lancer-tests.ps1 PE-01        # un seul test, par son identifiant
node tests/system-tests.mjs           # tests systeme de bout en bout
node tests/exploratoire.mjs           # tests exploratoires
.\tests\maintenance-sauvegarde.ps1    # test de maintenance
npx newman run tests/postman/EasyVote.postman_collection.json
```

6.2 Techniques statiques

Les techniques statiques n'exigent pas l'exécution du logiciel : elles trouvent les **causes** des défauts plutôt que les défaillances, et plus tôt, donc à moindre coût.

6.2.1 Place des techniques statiques dans le projet

Le tableau ci-dessous rapproche les techniques statiques du cours de leur usage dans EasyVote. Elles complètent les tests dynamiques : aucune des deux familles ne remplace l'autre.

Table 6.2 – Techniques statiques du cours et application dans EasyVote

Technique (cours)	Application dans EasyVote
Revue informelle (entre pairs)	Relecture par un second membre avant le passage d'un ticket Jira de *In Review* à *Done*
Revue technique (formelle)	Revue du code contre le cahier des charges, avec checklist et constats numérotés RV-01 à RV-06 (ci-dessous)
Inspection	Non pratiquée : elle exige un modérateur formé et des critères d'entrée et de sortie formels, hors du périmètre de ce projet
Analyse statique outillée	SonarQube (backend) ; ESLint, tsc et npm audit (frontend)

Critère de sortie de la revue : chaque constat est consigné (RV-xx) et rattaché à un défaut DEF-xx ou à une observation.

6.2.2 Revue technique

La revue technique (formelle) a été menée selon le processus : planification, lancement, préparation individuelle, réunion de revue, re-travail et suivi. Les rôles (modérateur, auteur, réviseur, scribe) tournent entre les trois membres. La checklist a été dérivée du cahier des charges.

Table 6.3 – Résultats de la revue technique du code

Réf.	Point de contrôle (CDC)	Constat dans le code	Défaut
RV-01	Access token valide 1 h (§4.4.1)	`application.properties`, ligne 23 : `easyvote.jwt.expiration=86400000` (24 h)	DEF-06
RV-02	Rôle CANDIDATE après validation (§3.1.2)	`CandidateService`, ligne 71 : rôle modifié dès le dépôt	DEF-02
RV-03	Photo $\leq$ 5 Mo (§3.1.2)	`PhotoUploadService`, ligne 34 : limite `5_000_000` octets ($\neq$ 5 242 880)	DEF-01
RV-04	L'admin ne vote pas (§6.1)	`VoteService.castVote()` : aucun contrôle de rôle	DEF-05
RV-05	Aucun lien SQL électeur $\leftrightarrow$ vote (§4.4.2)	même `token_hash` dans `votes` et `vote_tokens`	DEF-00
RV-06	Aucun secret dans le dépôt	`application.properties` : identifiants SMTP en clair	Observation

Les constats RV-02, RV-04 et RV-05 ont ensuite été **confirmés dynamiquement** (sections 6.9.1 et 6.9.2) : la revue a orienté les tests dynamiques.

6.2.3 Analyse statique avec SonarQube (backend)

SonarQube 10.6 (édition Community, lancé dans Docker) a analysé les 4 875 lignes de code Java du backend, en important la couverture JaCoCo des tests JUnit. L'analyse se relance avec `tests/sonar-analyse.ps1`.

Table 6.4 – Résultats de l'analyse SonarQube

Mesure	Valeur	Interprétation
Quality Gate (Sonar way)	Passed	première analyse : la règle porte sur le « nouveau code »
Vulnérabilités	0	note de sécurité A
Bugs	3	tous des **faux positifs** (voir ci-dessous)
Security hotspots	2	analysés et justifiés ci-dessous ; statut « Reviewed » non renseigné dans l'outil (0 %)
Code smells	45	note de maintenabilité A
Couverture	35,3 %	lignes 38,0 % et branches 25,3 % (JaCoCo, 38 tests JUnit)
Duplication	2,7 %	faible
Complexité cyclomatique	460	`ElectionService` 56, `ChatbotRulesService` 39, `VoteService` 22

Tri des résultats (étape indispensable d'une analyse statique) :

— **3 bugs (règle S6857, « placeholder mal formé ») = faux positifs.** Les annotations visées, par exemple `@Value("${easyvote.app.url:http://localhost:5173}")`, ont une valeur par défaut contenant une URL avec « : ». Spring utilise le premier « : » comme séparateur : le code fonctionne.

— **Hotspot « CSRF désactivé »** (`SecurityConfig`) : justifié, car l'API est stateless et authentifiée par JWT, ce que prévoit le cahier des charges (§5.2).

— **Hotspot « expression régulière »** (`CorsConfig`) : sans risque, l'expression ne s'applique qu'à une valeur de configuration et jamais à une saisie utilisateur.

— **Complexité cognitive** : `ChatbotRulesService.tryMatchRule()` atteint 37 pour un maximum recommandé de 15. Cette méthode, une longue suite de règles évaluées dans l'ordre, est justement celle où l'ancien test UT-CB3 a trouvé DEF-04 : la complexité signalée par l'outil et le défaut trouvé par le test dynamique se confirment mutuellement.

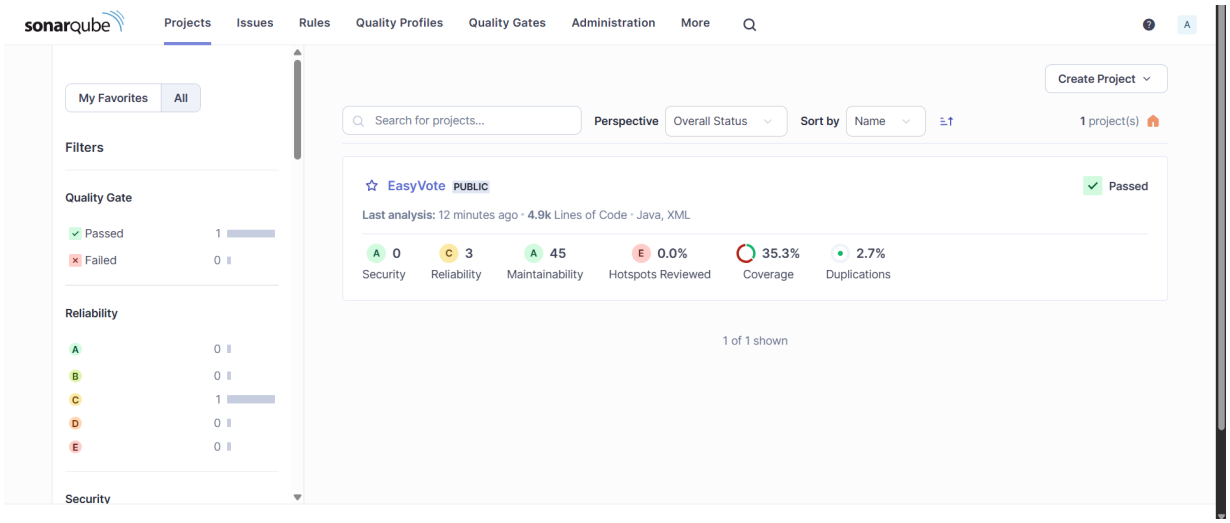


FIGURE 6.1 – SonarQube — projet EasyVote : Quality Gate réussi, 0 vulnérabilité, 3 bugs, 45 code smells, couverture 35,3 %, duplication 2,7 %

6.2.4 Analyse statique outillée du frontend

TABLE 6.5 – Résultats de l'analyse statique du frontend

Réf.	Outil	Commande	Résultat
SA-01	TypeScript	`npx tsc -b`	0 erreur de typage
SA-02	ESLint	`npx eslint .`	64 erreurs, 5 avertissements (66 fichiers)
SA-03	npm audit	`npm audit`	35 vulnérabilités (1 critique, 26 hautes, 6 moyennes, 2 basses)

Parmi les erreurs ESLint : 30 `no-explicit-any` (typage affaibli) et surtout **4 `rules-of-hooks`** dans `ChatbotWidget.tsx` : un `return null` conditionnel (ligne 168) précède trois `useEffect` et un `useCallback`. Si la condition change entre deux rendus, React lève une erreur. Le composant n'étant monté que dans le tableau de bord, la défaillance ne se produit pas aujourd'hui : c'est un **défaut latent** (DEF-07) que seule l'analyse statique pouvait détecter.

6.3 Techniques dynamiques boîte noire

Ces techniques dérivent les cas de test des **spécifications** (cahier des charges), sans regarder le code. Elles sont automatisées en JUnit (dossier `src/test/.../boitenoire`).

6.3.1 Partition d'équivalence — code OTP

Règle : le code OTP contient exactement 6 chiffres. Un représentant par classe suffit.

Table 6.6 – Partitions d'équivalence du code OTP

Réf.	Partition	Valeur	Attendu	Résultat
PE-01	Valide (6 chiffres)	`123456`	Accepté	**Réussi**
PE-02	Invalide : longueur ≠ 6	`12345`	Rejeté	**Réussi**
PE-03	Invalide : non numérique	`12AB56`	Rejeté (« uniquement des chiffres »)	**Réussi**

6.3.2 Analyse des valeurs limites — candidature

Règles : slogan ≤ 200 caractères, photo ≤ 5 Mo. On teste sur la limite et juste après.

Table 6.7 – Valeurs limites des champs de candidature

Réf.	Champ	Valeur	Attendu	Résultat
VL-01	Slogan	200 caractères (limite)	Accepté	**Réussi**
VL-02	Slogan	201 caractères (limite + 1)	Rejeté	**Réussi**
VL-03	Photo	5 × 1024 × 1024 octets	Acceptée	**Échec** (DEF-01)

VL-03 échoue avec le message « Fichier trop volumineux (max 5MB) » : le code compare la taille à 5 000 000 octets alors que 5 Mo = 5 242 880 octets. Une valeur moyenne (2,5 Mo) n'aurait jamais révélé ce défaut de frontière.

6.3.3 Table de décision — vote d'un étudiant

Conditions : C1 = élection `VOTE_ACTIVE`; C2 = l'étudiant n'a pas déjà voté.

Table 6.8 – Table de décision du vote

Règle	C1	C2	Action attendue	Résultat
R1	V	V	Vote enregistré + code de 10 caractères `[A-Z0-9]`	**Réussi**
R2	F	–	Refus « Le vote n'est pas ouvert », aucun vote enregistré	**Réussi**
R3	V	F	Refus « Vous avez déjà voté », aucun vote enregistré	**Réussi**

6.3.4 Test de transition d'état — cycle de vie d'une élection

États : `DRAFT` → `CAMPAIGN_ACTIVE` → `VOTE_ACTIVE` → `CLOSED`.

Table 6.9 – Transitions d'état de l'élection

Réf.	Transition	Attendu	Résultat
ST-20	DRAFT → CAMPAIGN_ACTIVE	Acceptée	**Réussi**
ST-21	CAMPAIGN_ACTIVE → VOTE_ACTIVE	Acceptée	**Réussi**
ST-23	DRAFT → VOTE_ACTIVE (saut d'état)	Rejetée, statut inchangé	**Réussi**

6.3.5 Test des cas d'utilisation — UC-03 « Voter »

Le cas d'utilisation est joué de bout en bout au niveau système (section 6.7) et manuellement en acceptation (UAT-01).

TABLE 6.10 – Cas d'utilisation UC-03 : Voter

Étape	Action	Résultat attendu	Résultat
1	Se connecter	JWT délivré	**Réussi**
2	Ouvrir l'élection `VOTE_ACTIVE`	Liste des candidats validés	**Réussi**
3	Choisir un candidat et cocher « Je comprends que mon vote est définitif »	Confirmation demandée	À exécuter
4	Confirmer	Vote enregistré + code de 10 caractères	**Réussi**
5	Consulter la participation	Statistiques visibles, choix non révélé	**Réussi**
4b	Voter une 2e fois (scénario alternatif)	Refus « déjà voté »	**Réussi**

6.4 Techniques dynamiques boîte blanche

Ces techniques dérivent les tests de la **structure du code réel** (dossier `src/test/.../boiteblanche`). La couverture obtenue est mesurée par JaCoCo.

6.4.1 Couverture des instructions — `hashToken()`

```
public String hashToken(String rawToken) {
    try {
        byte[] hash = MessageDigest.getInstance("SHA-256")
                .digest(rawToken.getBytes(StandardCharsets.UTF_8));
        StringBuilder sb = new StringBuilder();
        for (byte b : hash) sb.append(String.format("%02x", b));
        return sb.toString();
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException("Erreur de hashage SHA-256 introuvable", e);
    }
}
```

TABLE 6.11 – Couverture des instructions de `hashToken()`

Réf.	Entrée	Attendu	Résultat
UT-01a	`"abc"`	`ba7816bf...f20015ad` (vecteur officiel NIST)	**Réussi**
UT-01b	jeton aléatoire	64 caractères hexadécimaux, déterministe, ≠ jeton brut	**Réussi**
UT-01c	deux générations	deux jetons bruts différents	**Réussi**

Toutes les instructions atteignables sont couvertes. Le bloc `catch` est **inatteignable** (SHA-256 existe toujours dans la JVM) : 100 % des instructions est impossible ici, ce qui doit être justifié plutôt que forcé.

6.4.2 Couverture des branches — `generateTokensForElection()`

```
Election election = electionRepository.findById(electionId).orElseThrow(...); // B1
if (voteTokenRepository.existsByElectionId(electionId)) { return; }          // B2
for (User user : userRepository.findByIsVerifiedTrue()) {                     // B3
    String tokenHash = hashToken(generateRawToken());
    voteTokenRepository.save(VoteToken.builder()...tokenHash(tokenHash)...build());
}
```

TABLE 6.12 – Couverture des branches de la génération des jetons

Réf.	Branche	Données	Attendu	Résultat
B1	Élection introuvable	id inexistant	`ResourceNotFoundException`	**Réussi**
B2	Jetons déjà générés	`existsByElectionId` = vrai	aucun `save()` (idempotence)	**Réussi**
B3	Génération	3 étudiants vérifiés	3 jetons, hash de 64 caractères, tous distincts	**Réussi**

JaCoCo confirme **100 % des branches** de `VoteTokenService`.

6.4.3 Couverture des décisions — `validateDates()`

Trois décisions :

— D1 : `!campaignStart.isBefore(campaignEnd)`

— D2 : `campaignEnd.isAfter(voteStart)`

— D3 : `!voteStart.isBefore(voteEnd)`

TABLE 6.13 – Couverture des décisions de la validation des dates

Réf.	D1	D2	D3	Résultat attendu	Résultat
DC-01	F	F	F	Élection créée en DRAFT	**Réussi**
DC-02	V	–	–	Rejet « début de campagne »	**Réussi**
DC-03	F	F	V	Rejet « début de vote »	**Réussi**

D2 n'est évaluée qu'à FAUX : un quatrième cas serait nécessaire pour 100 % de couverture des décisions (limite assumée de l'échantillon).

6.4.4 Complexité cyclomatique — `castVote()`

La méthode `VoteService.castVote()` contient 11 points de décision : utilisateur absent, compte non vérifié, élection absente, candidat absent, candidat d'une autre élection, candidat non validé, statut ≠ `VOTE_ACTIVE`, fin de vote (2 conditions), jeton absent, jeton déjà utilisé.

$$\mathbf{V(G) = \text{nombre de décisions} + 1 = 11 + 1 = 12}$$

Le seuil habituel de 10 est dépassé (UT-07 en échec) : la méthode devrait être découpée. Les chemins indépendants sont répartis entre la table de décision, le MC/DC et les trois chemins suivants :

Table 6.14 – Chemins indépendants de `castVote()`

Réf.	Chemin	Attendu	Résultat
CC-1	sortie à la décision 2	compte non vérifié : `ForbiddenException`	**Réussi**
CC-2	sortie à la décision 5	candidat d'une autre élection : refus	**Réussi**
CC-3	jeton absent → généré à la volée	vote enregistré avec un hash de 64 caractères ; l'entité `Vote` n'a aucun champ `User`	**Réussi**

6.4.5 Couverture MC/DC — fin de la période de vote

```
if (election.getVoteEnd() != null && now.isAfter(election.getVoteEnd()))   // condition A,
    condition B
    throw new BusinessException("La période de vote est terminée");
```

Pour $n = 2$ conditions, MC/DC exige $n + 1 = 3$ tests où chaque condition change seule la décision.

Table 6.15 – Couverture MC/DC de la condition de fin de vote

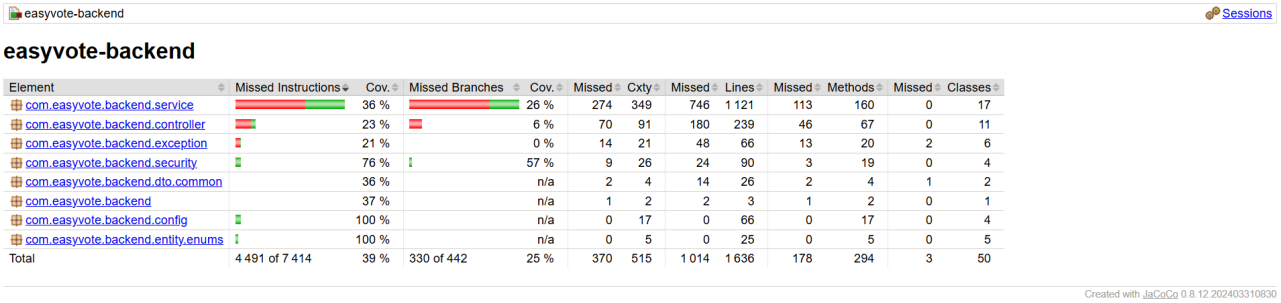
Test	A	B	Décision	Paire démontrée	Résultat
T1	V	V	V (vote refusé)	—	**Réussi**
T2	V	F	F (vote accepté)	(T1, T2) : B influe seul	**Réussi**
T3	F	–	F (vote accepté)	(T1, T3) : A influe seul	**Réussi**

6.4.6 Couverture mesurée par JaCoCo

Table 6.16 – Couverture de code mesurée (JaCoCo)

Classe	Lignes	Branches
VoteTokenService	84,2 %	100 %
VoteService	66,3 %	60,7 %
VoteAnalyticsService	57,4 %	35,0 %
ElectionService	56,0 %	42,2 %
JwtUtil	66,0 %	50,0 %
ChatbotRulesService	34,1 %	30,3 %
Projet complet	38,0 %	25,3 %

La couverture globale est faible car seuls les modules critiques ont été ciblés ; elle est élevée là où le risque est le plus fort (jetons et vote).


easyvote-backend

Sessions

easyvote-backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.easyvote.backend.service		36 %		26 %	274	349	746	1 121	113	160	0	17
com.easyvote.backend.controller		23 %		6 %	70	91	180	239	46	67	0	11
com.easyvote.backend.exception		21 %		0 %	14	21	48	66	13	20	2	6
com.easyvote.backend.security		76 %		57 %	9	26	24	90	3	19	0	4
com.easyvote.backend.dto.common		36 %		n/a	2	4	14	26	2	4	1	2
com.easyvote.backend		37 %		n/a	1	2	2	3	1	2	0	1
com.easyvote.backend.config		100 %		n/a	0	17	0	66	0	17	0	4
com.easyvote.backend.entity.enums		100 %		n/a	0	5	0	25	0	5	0	5
Total	4 491 of 7 414	39 %	330 of 442	25 %	370	515	1 014	1 636	178	294	3	50

Created with JaCoCo 0.8.12.202403310830

FIGURE 6.2 – Rapport JaCoCo — couverture par paquet : 39 % des instructions et 25 % des branches (1 636 lignes, 38 % couvertes)

6.5 Tests unitaires : chatbot et non-régression

TABLE 6.17 – Tests unitaires du chatbot EasyBot et de non-régression

Réf.	Type	Vérification	Résultat
UT-01 (chatbot)	Fonctionnel	« Bonjour » → message de bienvenue	**Réussi**
UT-06	Non fonctionnel	réponse de la couche règles en moins de 50 ms	**Réussi**
UT-M01	Régression	jeu de 10 votes de référence (5, 3 et 2 voix) sur 20 électeurs : classement, pourcentages (50, 30 et 20 %) et participation (50 %) inchangés	**Réussi**

UT-M01 appelle `VoteAnalyticsService.getResults()` avec des dépôts simulés ; les candidats sont fournis dans le désordre pour vérifier aussi le tri. Les valeurs attendues sont figées : après un refactoring, le résultat doit rester identique. Dans le script `lancer-tests.ps1`, le test chatbot « Bonjour » porte l'identifiant UT-01, à ne pas confondre avec UT-01a à UT-01c (`hashToken()`) du plan.

Le test UT-CB3 (« Comment devenir candidat ? »), qui avait révélé le défaut DEF-04, a été retiré de la suite automatisée. Réponse obtenue à l'époque : « Il n'y a pas d'élection en cours, donc pas de liste de candidats. » Les règles étant évaluées dans l'ordre, la règle « candidat » est déclenchée avant la règle « devenir candidat ». **Le défaut n'est pas corrigé** : il reste ouvert dans la liste des défauts.

6.6 Tests d'intégration et tests d'API

6.6.1 Tests d'intégration automatisés (JUnit + MockMvc + H2)

L'application Spring démarre réellement (contrôleurs, filtre JWT, services, JPA) sur une base H2 en mémoire ; seul l'envoi d'email est simulé.

Table 6.18 – Tests d'intégration exécutés

Réf.	Scénario	Vérifications	Résultat
IT-01	POST `/api/auth/register`	HTTP 201 ; compte STUDENT non vérifié ; mot de passe haché BCrypt ; OTP à 6 chiffres envoyé	**Réussi**
IT-09	GET `/api/admin/users`	sans jeton → 401 ; jeton STUDENT → 403	**Réussi**
IT-04	POST `/api/elections/{id}/vote`	HTTP 201 et code de 10 caractères ; pas de colonne `user_id` ; aucune jointure ne relie un électeur à son vote	**Échec** (DEF-00)
IT-07	GET `/api/elections`	5 élections ; 20 appels successifs après un appel de chauffe ; temps de réponse moyen ≤ 200 ms	**Réussi**

6.6.2 Tests d'API avec Postman

Une collection Postman (`tests/postman/EasyVote.postman_collection.json`) automatise les vérifications sur l'API réelle. Le jeton obtenu par PM-01 est stocké dans une variable et réutilisé par PM-03. La collection est exécutable dans Postman (*Run collection*) ou en ligne de commande avec Newman.

Table 6.19 – Tests Postman

Réf.	Requête	Type	Assertions	Résultat
PM-01	POST `/api/auth/login`	Fonctionnel	statut 200 ; jeton renvoyé ; rôle ADMIN	**Réussi**
PM-03	GET `/api/auth/me` (Bearer)	Fonctionnel	statut 200 ; profil du jeton ; mot de passe jamais renvoyé	**Réussi**
PM-02	GET `/api/admin/users` sans jeton	Non fonct. (sécurité)	statut 401 ; message « Authentification requise » ; réponse < 200 ms	**Réussi**

Listing 6.1 – Script de test Postman de PM-01

```
pm.test("Statut HTTP 200", function () {
    pm.response.to.have.status(200);
});
const json = pm.response.json();
pm.test("Le rôle de l'utilisateur est ADMIN", function () {
    pm.expect(json.data.user.role).to.eql("ADMIN");
});
pm.collectionVariables.set("token", json.data.accessToken);  // réutilisé par PM-03
```

Listing 6.2 – Exécution de la collection avec Newman

```
-> PM-01 Connexion administrateur (fonctionnel)
  POST http://localhost:8082/api/auth/login [200 OK, 90ms]
  v  Statut HTTP 200
  v  Un jeton JWT est renvoyé
  v  Le rôle de l'utilisateur est ADMIN
-> PM-03 Profil connecté /auth/me avec le jeton (fonctionnel)
  GET http://localhost:8082/api/auth/me [200 OK, 9ms]
  v  Statut HTTP 200
  v  Le profil renvoyé est celui du jeton (ahmed@test.com, ADMIN)
  v  Le mot de passe n'est jamais renvoyé
-> PM-02 Route admin protégée sans jeton (non fonctionnel : sécurité)
  GET http://localhost:8082/api/admin/users [401 Unauthorized, 4ms]
```

```
  v  Accès refusé : statut HTTP 401
  v  Message « Authentification requise »
  v  Réponse en moins de 200 ms (CDC 5.1)
assertions : 9 exécutées, 0 en échec
```

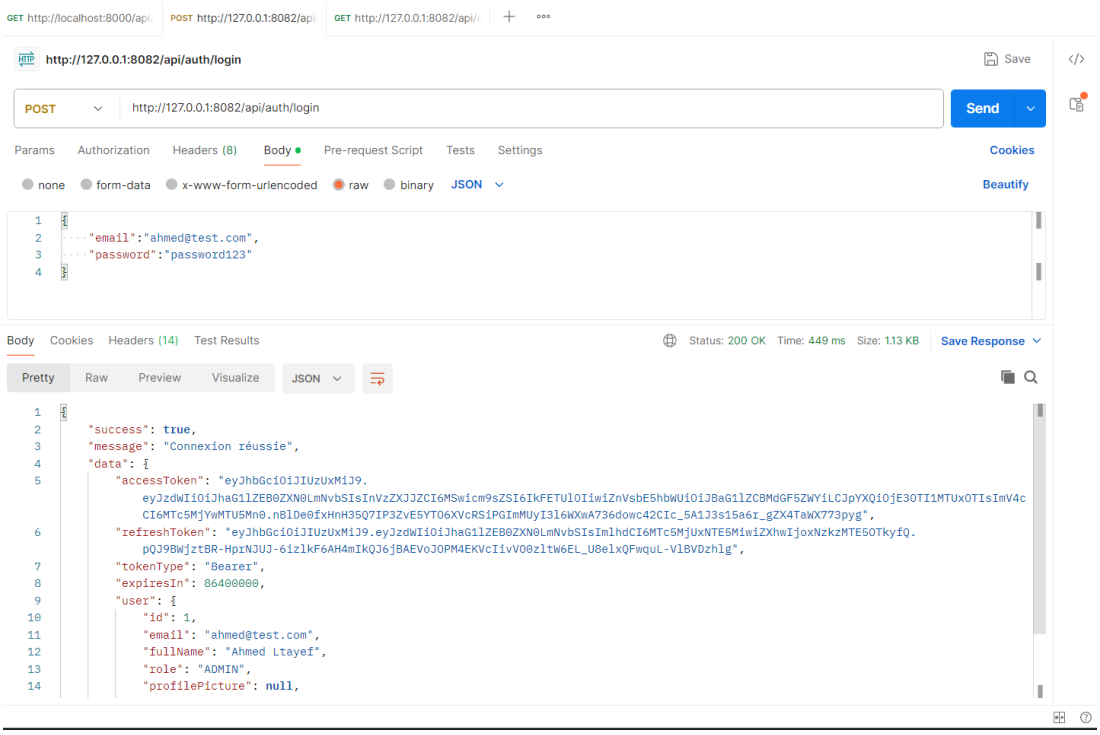


FIGURE 6.3 – Postman — PM-01 : `POST /api/auth/login` avec le compte administrateur, 200 OK ; le jeton d'accès, le jeton de rafraîchissement et `expiresIn` (86 400 000 ms, soit 24 h) sont renvoyés

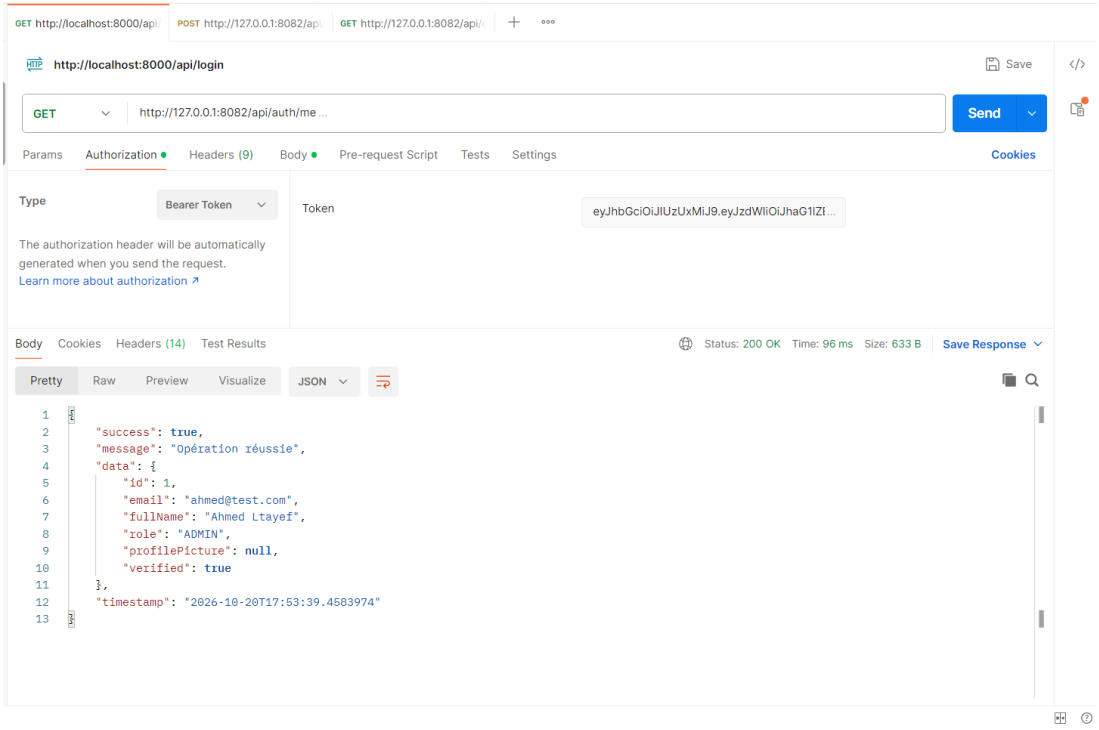


FIGURE 6.4 – Postman — PM-03 : `GET /api/auth/me` avec le jeton en *Bearer Token*, 200 OK en 96 ms ; le profil est celui du jeton et le mot de passe n'est pas renvoyé

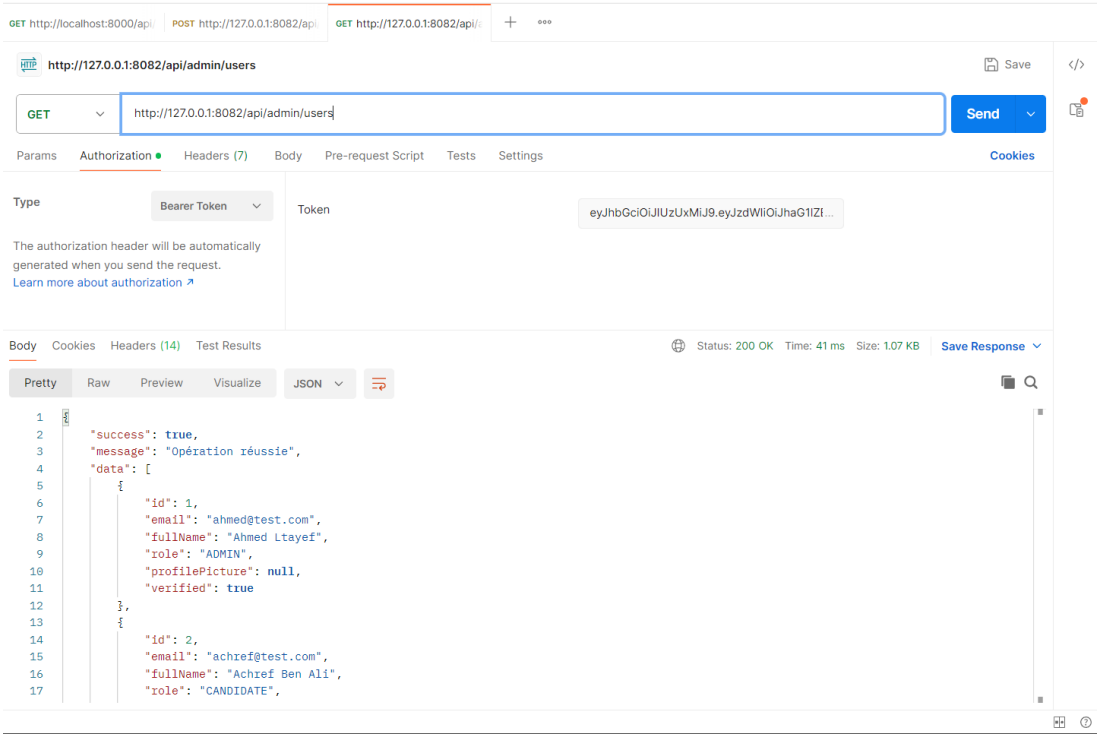


FIGURE 6.5 – Postman — `GET /api/admin/users` avec le jeton ADMIN, 200 OK en 41 ms : la contrepartie de PM-02, qui renvoie 401 sans jeton

6.6.3 Vérification manuelle du JWT avec jwt.io (JWT-01)

Le jeton renvoyé par `/api/auth/login` a été décodé sur `https://jwt.io` pour vérifier son contenu par rapport au cahier des charges (§4.4.1 : access token valide 1 h).

Listing 6.3 – Contenu décodé d'un access token EasyVote

```
HEADER  : { "alg": "HS512" }
PAYLOAD : { "sub": "ahmed@test.com", "userId": 1, "role": "ADMIN",
            "fullName": "Ahmed Ltayef", "iat": 1792444682, "exp": 1792531082 }
exp - iat = 86400 s = 24 h
```

— **Conforme :** l'algorithme de signature (HS512), l'identité (`sub`, `userId`) et le rôle sont présents et cohérents avec `/auth/me`.

— **Non conforme :** la durée de validité est de **24 h** au lieu de 1 h. Le défaut DEF-06, trouvé en revue (RV-01), est ainsi **confirmé dynamiquement**.

— **Observation :** le payload d'un JWT est seulement encodé (Base64), pas chiffré : l'email et le nom complet sont lisibles par quiconque possède le jeton.

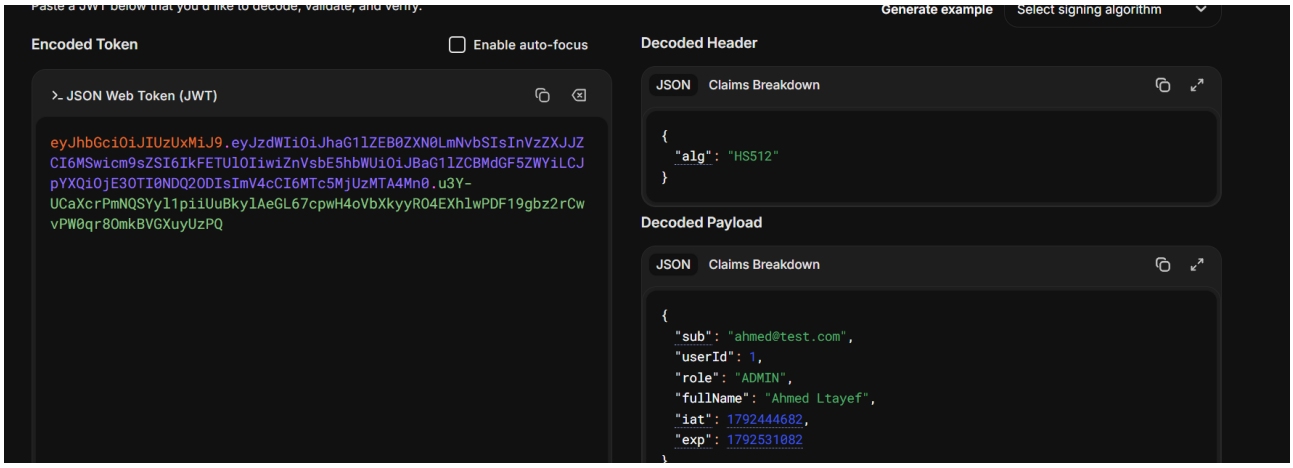


FIGURE 6.6 – jwt.io — access token décodé : algorithme HS512, rôle ADMIN, `iat` = 1792444682 et `exp` = 1792531082 (écart de 86 400 s, soit 24 h)

6.7 Tests système de bout en bout

Le script `tests/system-tests.mjs` pilote l'application complète (backend + MariaDB) comme le ferait l'interface. Il joue le scénario de démonstration du cahier des charges (§7.3) et le cas d'utilisation UC-03.

Listing 6.4 – Exécution des tests système

```
ST-01 Scénario complet : création -> candidature -> validation -> vote -> résultats
  [OK] ST-01.1 l'admin crée une élection (DRAFT)
  [OK] ST-01.2 ouverture de la campagne
  [OK] ST-01.3 un étudiant postule (PENDING)
  [OK] ST-01.4 l'admin valide la candidature
  [OK] ST-01.5 ouverture du vote + génération des jetons
  [OK] ST-01.6 l'étudiant vote -> code de confirmation « UNVEB634QL »
ST-02 Interdiction du double vote
```

```
  [OK] ST-02 le 2e vote est refusé avec un message clair
ST-03 Transparence : participation pendant le vote, détails après clôture
  [OK] ST-03.1 pendant le vote, l'étudiant ne voit PAS les résultats détaillés
  [OK] ST-03.2 pendant le vote, l'étudiant voit la participation
  [OK] ST-03.3 après clôture, résultats détaillés publiés (rang 1 = 100 %)
  [OK] ST-05 export du rapport PDF officiel
ST-NF Performance
  [OK] ST-NF GET /elections : moyenne 61.1 ms sur 20 appels (< 200 ms)
Bilan : 12 vérifications réussies, 0 en échec.
```

Une 13e vérification, **REG-002** (non-régression du calcul des résultats : après la clôture, classement, votes, pourcentages et participation sont identiques entre deux lectures de `/results`), a été ajoutée au script après cette exécution. Elle est À exécuter : elle demande le backend et les comptes de test chargés.

6.8 Tests de charge et de performance avec JMeter

Apache JMeter 5.6.3 exécute le plan `tests/jmeter/EasyVote-charge.jmx` (ST-08). Chaque utilisateur virtuel se connecte une fois (`POST /api/auth/login`, extraction du jeton JWT), puis consulte en boucle la liste des élections et les statistiques de participation, avec une pause de 300 ms. Deux assertions sont appliquées à chaque requête : code HTTP 200 et temps de réponse inférieur à **200 ms** (CDC §5.1). Le test se lance avec `tests/jmeter-charge.ps1 -Users N` et produit un rapport HTML.

Table 6.20 – Résultats JMeter selon le nombre d'utilisateurs simultanés

Scénario	Requêtes	HTTP 200	Plus de 200 ms	Moyenne	90e centile
50 utilisateurs × 10 boucles	1 050	100 %	0 %	12 ms	14 ms
200 utilisateurs × 5 boucles	2 200	100 %	0 %	14 ms	20 ms
500 utilisateurs × 5 boucles	5 500	100 %	51,8 %	486 ms	1 172 ms

- **Fonctionnel sous charge :** aucune erreur HTTP, même à 500 utilisateurs (5 500 réponses 200).
- **Performance :** l'exigence de 200 ms est respectée jusqu'à 200 utilisateurs, mais **pas à 500** : 2 847 requêtes dépassent 200 ms, et la connexion (hachage BCrypt) monte à 593 ms en moyenne. C'est la non-conformité **DEF-08**.
- **Limite de la mesure :** le test a été exécuté sur un poste de développement, avec JMeter, le backend et la base sur la même machine. Le résultat est à confirmer sur un serveur dédié avant toute conclusion définitive.

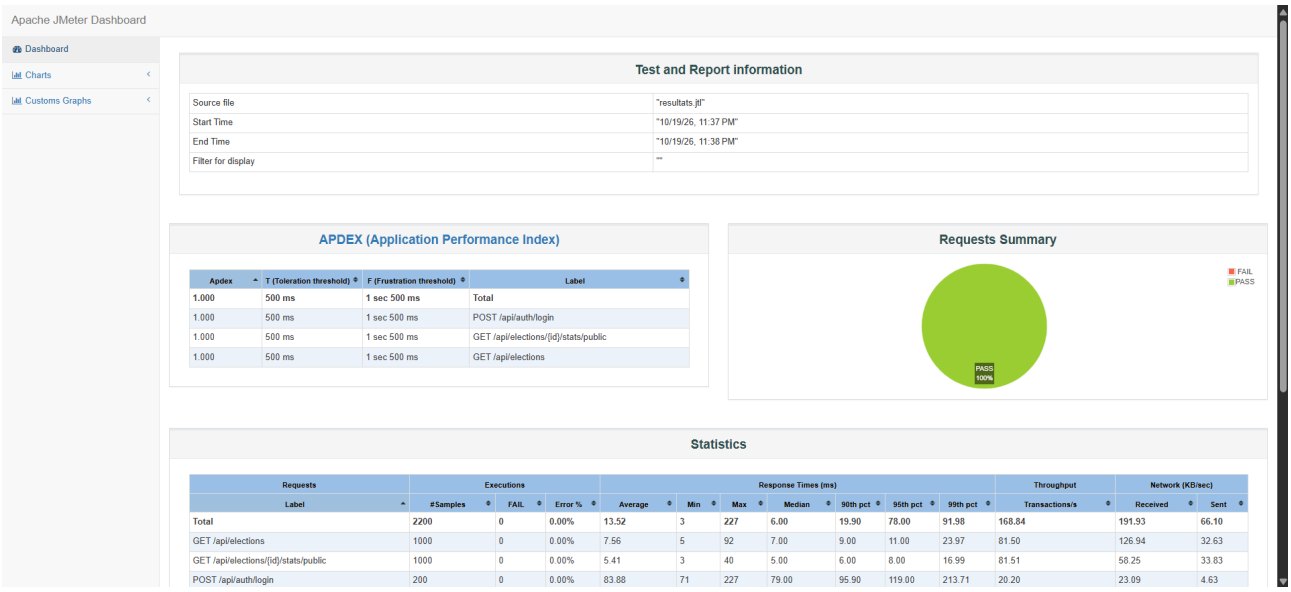


FIGURE 6.7 – Rapport JMeter — scénario à 200 utilisateurs : 2 200 requêtes, 0 erreur, temps moyen 13,5 ms, 90e centile 19,9 ms

6.9 Tests manuels

6.9.1 Vérification SQL de l'anonymat (SQL-01)

Le plan initial vérifiait seulement l'absence de colonne `user_id` dans la table `votes`. Ce test passe, mais il ne prouve pas l'exigence du cahier des charges (§4.4.2) : « aucune requête SQL ne peut établir de lien entre un utilisateur et son vote ». Or la table `votes` stocke le même `token_hash` que la table `vote_tokens`, qui contient `user_id`. Sur la base réelle :

Listing 6.5 – Requête SQL de ré-identification d'un vote

```
SELECT u.email AS electeur, cu.full_name AS a_vote_pour
FROM votes v
JOIN vote_tokens t ON t.token_hash = v.token_hash
JOIN users u       ON u.id = t.user_id
JOIN candidates c  ON c.id = v.candidate_id
JOIN users cu      ON cu.id = c.user_id;

electeur          a_vote_pour
talel@test.com    Achref Ben Ali
```

N'importe quel administrateur ayant accès à la base peut donc savoir pour qui chaque étudiant a voté : c'est le **défaut critique DEF-00**. Le test d'intégration IT-04 a été renforcé avec cette jointure et échoue désormais, comme il le doit. **Leçon :** un test doit vérifier l'exigence, pas l'implémentation supposée.

Piste de correction : ne stocker dans `votes` aucune valeur reliable au jeton (ou un hash salé avec un secret différent), et ne conserver dans `vote_tokens` que l'indicateur « déjà voté ».

6.9.2 Tests exploratoires

Guidés par la revue technique, ces tests vérifient trois règles du cahier des charges sur l'application en fonctionnement (rejoués par `tests/exploratoire.mjs`).

Table 6.21 – Tests exploratoires

Réf.	Action	Attendu (CDC)	Obtenu
EX-1	Postuler à une élection encore en DRAFT	Refus (§3.1.2)	HTTP 201 : **Échec** (DEF-03)
EX-2	Lire `/api/auth/me` avec une candidature PENDING	Rôle STUDENT	CANDIDATE : **Échec** (DEF-02)
EX-3	Voter avec le compte administrateur	Refus (§6.1)	Vote enregistré : **Échec** (DEF-05)

6.9.3 Tests d'acceptation automatisés

Un cas automatisable par type de test a été écrit dans `AcceptationTest` (JUnit, MockMvc, H2) : le scénario est joué comme le ferait l'utilisateur, par l'API, avec une connexion réelle et non un jeton fabriqué.

Table 6.22 – Tests d'acceptation automatisés

Réf.	Type	Vérification	Résultat
UAT-01	Fonct.	étudiant : connexion → consultation des élections → vote avec code de confirmation, sans blocage	**Réussi**
UAT-06	Non-fonct.	le PDF des résultats est un vrai document (en-tête `%PDF`, taille $> 1\,000$ octets) ; sa clarté reste à juger par un lecteur	**Réussi**
UAT-07	Struct.	les trois rôles accèdent à `/auth/me` et `/elections` ; `/admin/users` répond 200 à l'ADMIN et 403 au STUDENT et au CANDIDATE	**Réussi**
UAT-M03	Régr.	création et pilotage d'une élection : 401 pour un anonyme, 403 pour un étudiant, autorisé pour l'ADMIN	**Réussi**

6.9.4 Tests d'acceptation manuels (UAT)

Les autres tests d'acceptation sont exécutés manuellement dans le navigateur, chaque membre jouant un rôle et notant la facilité d'utilisation de 1 à 5.

Table 6.23 – Exécution des tests d'acceptation

Réf.	Testeur	Scénario	Résultat	Note /5
UAT-01	Rawen (étudiant)	connexion → programme → vote avec confirmation → code	[à compléter]	[_]
UAT-02	Issra (candidat)	Ma campagne → programme → annonce → réponse	[à compléter]	[_]
UAT-03	Talel (admin)	création → validation → vote → clôture → PDF	[à compléter]	[_]

6.9.5 Test de maintenance : sauvegarde et restauration

Le script `tests/maintenance-sauvegarde.ps1` sauvegarde la base (`mysqldump`), la restaure dans une copie et compare le contenu table par table, sans toucher à la base réelle.

Listing 6.6 – Exécution du test de maintenance

```
[OK]    MT-01 sauvegarde creee : tests\sauvegarde.sql
        users        original =   5   restauree =   5
        elections    original =   3   restauree =   3
        candidates   original =   3   restauree =   3
        votes        original =   3   restauree =   3
        vote_tokens  original =   7   restauree =   7
[OK]    MT-02 donnees restaurees a l'identique
```

6.9.6 Tests liés aux changements : confirmation et régression

La suite automatisée est **entièrement rejouable**. Après correction d'un défaut (par exemple DEF-01 : remplacer `5_000_000` par `5 * 1024 * 1024`), on exécute :

— un **test de confirmation** : le test VL-03 (`lancer-tests.ps1 VL-03`) doit passer au vert ;

— un **test de régression** : toute la suite (`lancer-tests.ps1`, puis `system-tests.mjs`) est relancée et tous les autres tests doivent rester verts.

6.10 Tests manuels et automatisés : synthèse

Table 6.24 – Répartition des tests exécutés par technique et par mode

Technique	Manuel	Automatisé
Statique	Revue technique (RV-01..06), tri des alertes SonarQube	SonarQube, ESLint, tsc, npm audit (SA-01..03)
Boîte noire	UAT-01..03, cas d'utilisation UC-03	PE, VL, R, ST-20/21/23 (12 tests JUnit)
Boîte blanche	Calcul de V(G)	UT-01, B, DC, CC, T (15 tests JUnit) + JaCoCo
Intégration / API	jwt.io (JWT-01)	IT-01/04/07/09 (JUnit), PM-01..03 (Postman)
Système	SQL-01, exploratoires EX-1..3	`system-tests.mjs` (12 vérifications), JMeter (ST-08)
Unitaire / régression	—	UT-01 (chatbot), UT-06, UT-M01 (3 tests JUnit)
Acceptation	UAT-01..03 en navigateur	UAT-01, 06, 07, M03 (4 tests JUnit)
Maintenance	—	MT-01, MT-02

6.11 Bilan de l'exécution

Table 6.25 – Bilan chiffré de l'exécution

Famille	Exécutés	Réussis	Échecs	Mode
Boîte noire (JUnit)	12	11	1	Automatisé
Boîte blanche (JUnit)	15	15	0	Automatisé
Unitaire (JUnit)	3	3	0	Automatisé
Intégration (JUnit)	4	3	1	Automatisé
Acceptation automatisée (JUnit)	4	4	0	Automatisé
API Postman (assertions)	9	9	0	Automatisé
Système E2E (vérifications)	12	12	0	Automatisé
Maintenance	2	2	0	Automatisé
jwt.io	1	0	1	Manuel
SQL anonymat	1	0	1	Manuel
Exploratoires	3	0	3	Manuel
Charge JMeter (scénarios)	3	2	1	Automatisé
SonarQube (règles)	1	1	0	Automatisé
Acceptation manuelle	3	[_]	[_]	Manuel

Au total, la suite JUnit compte **38 tests : 36 réussis et 2 en échec** (VL-03 et IT-04). Ces échecs sont voulus : ils signalent des défauts réels du code (DEF-01 et DEF-00).

Sur les **68 cas du plan** (Testiny) : 24 réussis, 6 en échec, 5 partiellement exécutés, 27 non exécutés et 6 tests d'acceptation à exécuter par l'équipe. Les cas non exécutés relèvent surtout de la maintenance (migrations, déploiement) et de la charge, qui nécessitent un environnement non disponible pour ce projet. L'exécution a en revanche ajouté des tests absents du plan initial (techniques boîte noire et boîte blanche détaillées, Postman, jwt.io, tests exploratoires), qui sont à reporter dans Testiny.

6.11.1 Défauts détectés

Table 6.26 – Défauts détectés pendant l'exécution

ID	Gravité	Description	Exigence	Détecté par
DEF-00	**Critique**	Le vote est ré-identifiable par jointure sur `token_hash`	EX-09	Revue, IT-04, SQL-01
DEF-05	Majeure	L'administrateur peut voter	EX-03	Revue, EX-3
DEF-06	Majeure	Access token valide 24 h au lieu de 1 h	EX-02	Revue, jwt.io
DEF-02	Majeure	Rôle CANDIDATE attribué avant la validation	EX-06	Revue, EX-2
DEF-03	Majeure	Candidature acceptée sur une élection DRAFT	EX-05	EX-1
DEF-01	Mineure	Photo de 5 Mo exactement rejetée	EX-05	Revue, VL-03
DEF-04	Mineure	Mauvaise réponse à « Comment devenir candidat ? »	EX-13	UT-CB3 (retiré de la suite)
DEF-08	Majeure	Temps de réponse > 200 ms pour 52 % des requêtes à 500 utilisateurs	EX-14	JMeter (ST-08)
DEF-07	Mineure (latent)	Hooks React appelés après un `return` conditionnel	—	ESLint

Chaque défaut est enregistré dans Testiny et rattaché au cas de test qui l'a révélé.

Conclusion

Ce travail a couvert l'ensemble des objectifs fixés : choix du projet EasyVote, expression des besoins sous forme de 15 exigences traçables, organisation du travail d'équipe avec **Jira**, gestion de 68 cas de test dans **Testiny**, élaboration d'un plan couvrant les quatre niveaux et les cinq types de tests, puis exécution de ce plan avec des techniques **statiques et dynamiques**, en mode **manuel et automatisé**, avec les outils **JUnit**, **Postman**, **SonarQube** et **JMeter**.

L'exécution montre que les techniques sont **complémentaires** : aucune n'aurait trouvé seule les neuf défauts.

- Les **valeurs limites** ont révélé un défaut de frontière invisible avec des valeurs moyennes (DEF-01).
- La **revue technique** a trouvé des écarts au cahier des charges et orienté les tests exploratoires (DEF-02, DEF-05, DEF-06).
- L'**analyse statique** (ESLint, SonarQube) a trouvé un défaut latent qu'aucun test dynamique ne déclenche (DEF-07) et a mesuré la qualité du code ; le tri de ses alertes a écarté 3 faux positifs.
- Le **test de charge JMeter** a montré que l'exigence de 200 ms tient jusqu'à 200 utilisateurs mais pas à 500 (DEF-08), ce qu'aucun test fonctionnel ne pouvait révéler.
- Les **tests d'intégration et SQL**, une fois correctement écrits, ont révélé le défaut le plus grave (DEF-00).
- Les techniques **boîte blanche** (JUnit + JaCoCo) ont prouvé, mesures à l'appui, la solidité du cœur cryptographique (100 % des branches de la génération des jetons).

Le scénario de démonstration fonctionne de bout en bout (12/12), ce qui satisfait le critère fonctionnel. En revanche, l'anonymat du vote, exigence centrale du projet, **n'est pas garanti** tant que DEF-00 n'est pas corrigé : c'est la priorité absolue, suivie des défauts de sécurité DEF-05 et DEF-06. Les prochaines étapes sont la correction de ces défauts, l'exécution des tests de confirmation et de régression, l'exécution des tests d'acceptation et l'intégration de la suite automatisée dans une chaîne d'intégration continue.